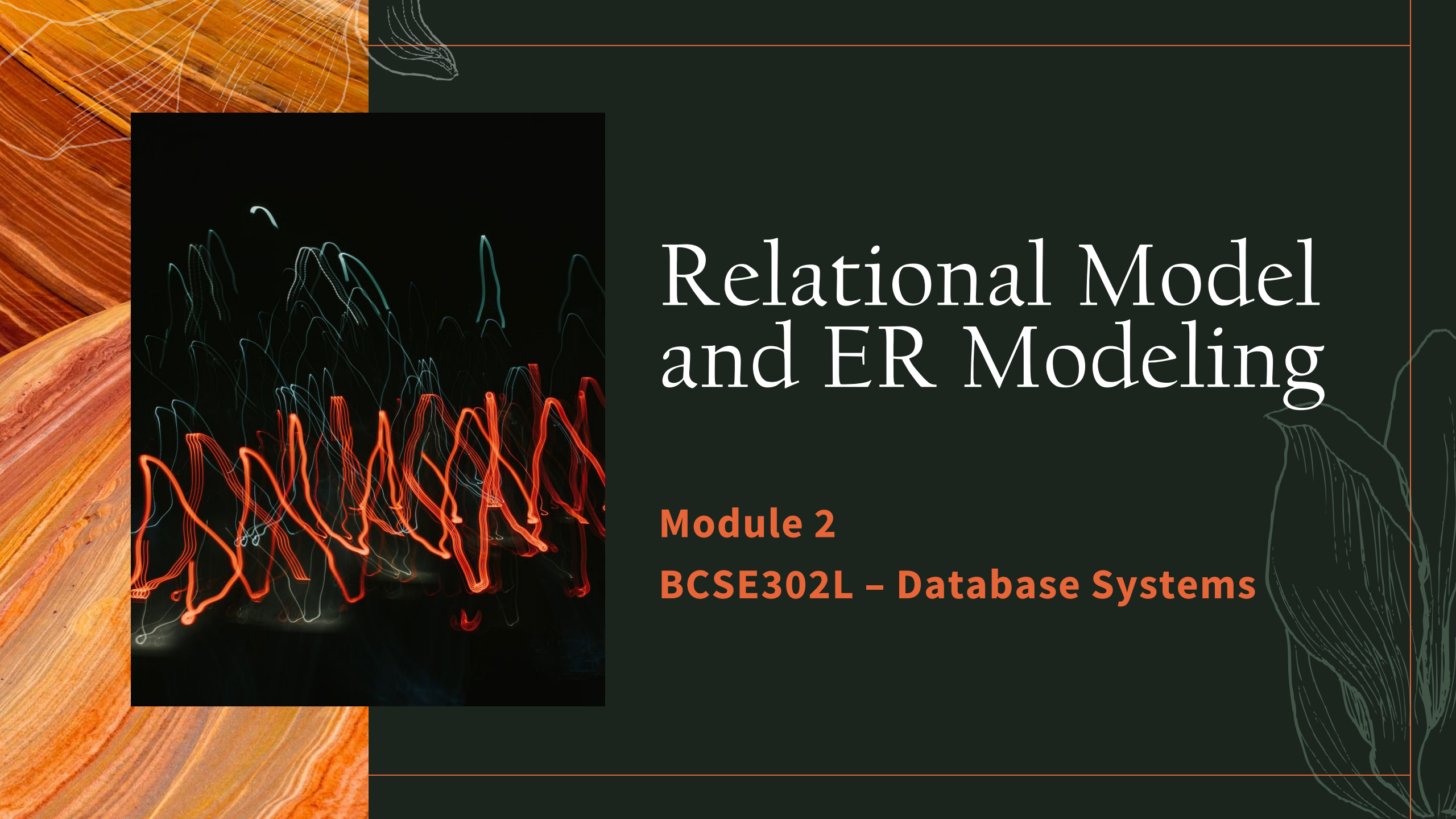




Relational Model and ER Modeling

Module 2

BCSE302L – Database Systems

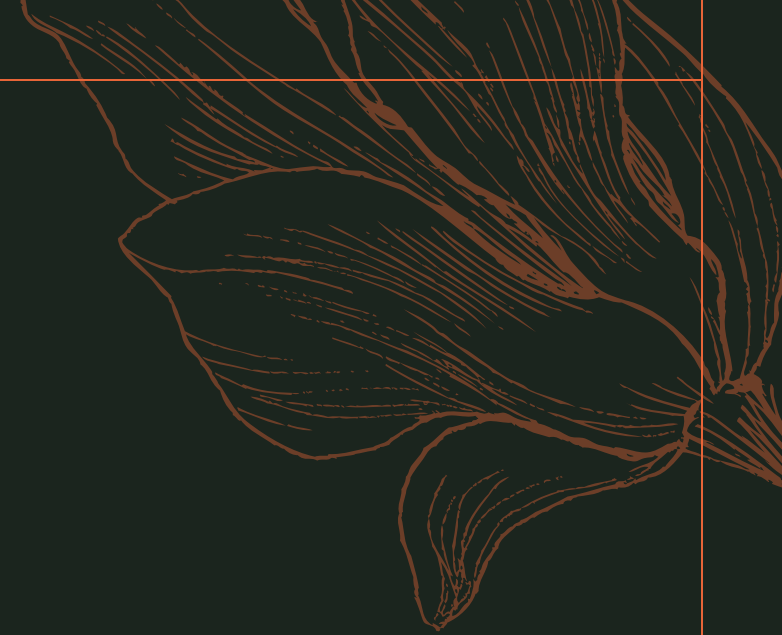
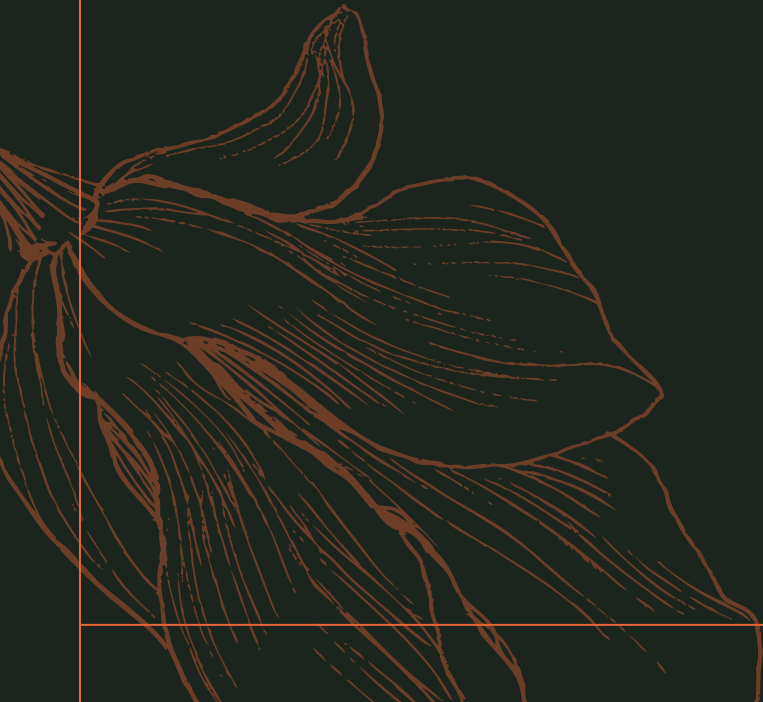
Agenda

- Entity Sets
- Relationship Sets
- Design Issues
- Mapping Constraints
- Keys
- E-R Diagram
- Extended E-R Features
- Design of an E-R Database Schema
- Reduction of an E-R Schema to Tables



Basic Concepts

Entity-Relationship Modeling



Entity Sets

- A database can be modeled as:
 - a collection of entities,
 - relationship among entities.
- An **entity** is an object that exists and is distinguishable from other objects.
 - Example: specific person, company, event, plant
- Entities have attributes
 - Example: people have names and addresses
- An **entity set** is a set of entities of the same type that share the same properties.
 - Example: set of all persons, companies, trees, holidays

Entity Sets – customer and loan

customer-id customer- customer- customer-
 name street city

321-12-3123	Jones	Main	Harrison
019-28-3746	Smith	North	Rye
677-89-9011	Hayes	Main	Harrison
555-55-5555	Jackson	Dupont	Woodside
244-66-8800	Curry	North	Rye
963-96-3963	Williams	Nassau	Princeton
335-57-7991	Adams	Spring	Pittsfield

customer

loan- amount
number

L-17	1000
L-23	2000
L-15	1500
L-14	1500
L-19	500
L-11	900
L-16	1300

loan

Attributes

- An entity is represented by a set of attributes, that is descriptive properties possessed by all members of an entity set.

Example: *customer = (customer-id, customer-name,
customer-street, customer-city)*

loan = (loan-number, amount)

- **Domain** – the set of permitted values for each attribute
- Attribute types:
 - Simple and composite attributes.
 - Single-valued and multi-valued attributes
 - E.g. multivalued attribute: phone-numbers
 - Derived attributes
 - Can be computed from other attributes
 - E.g. age, given date of birth

Composite Attributes

Composite
Attributes

name

first-name *middle-initial* *last-name*

address

street *city* *state* *postal-code*

Component
Attributes

street-number *street-name* *apartment-number*

Relationship Sets

- A **relationship** is an association among several entities

- Example:

Hayes depositor A-102
customer entity relationship set account entity

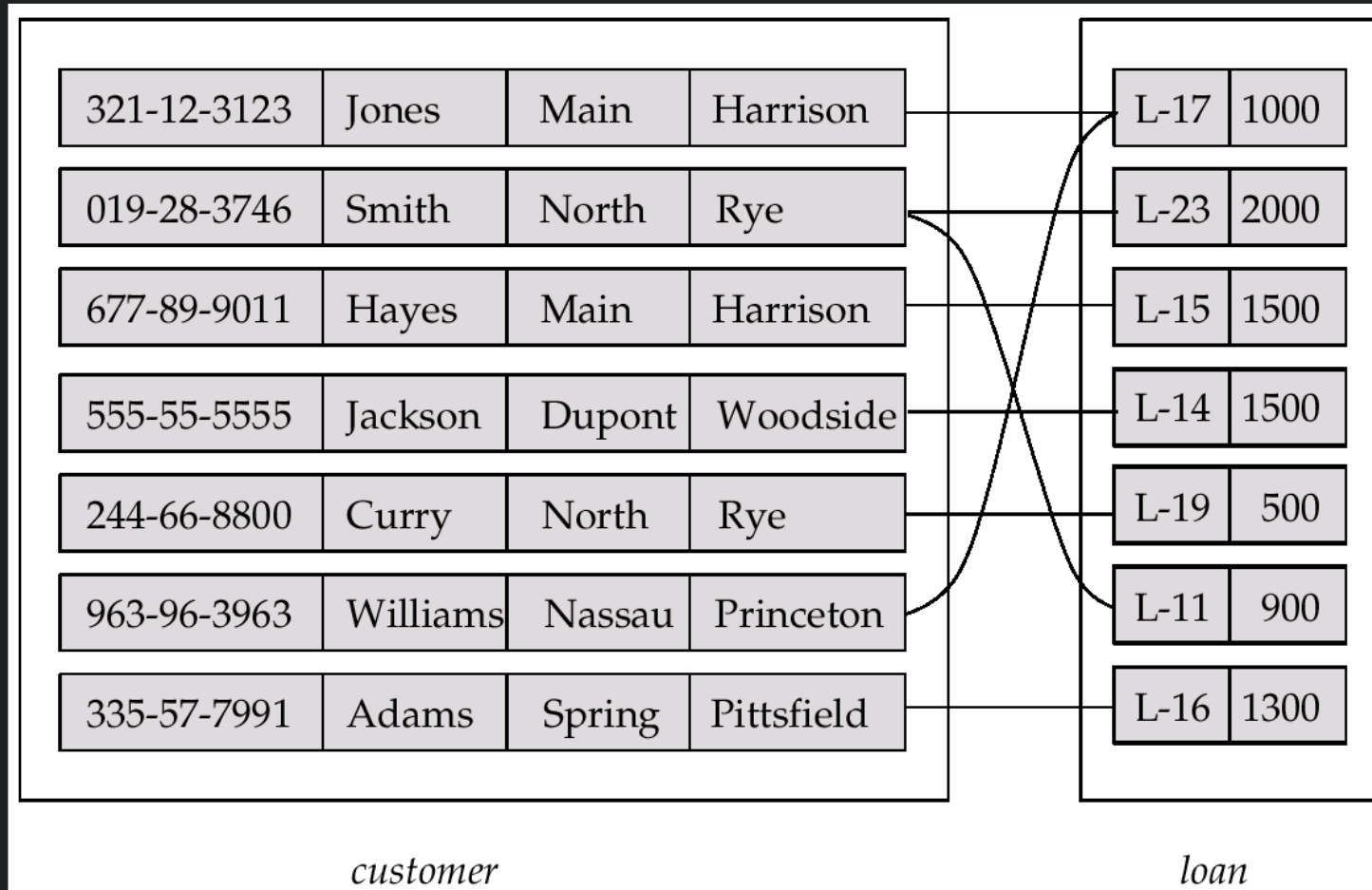
- A **relationship set** is a mathematical relation among $n \geq 2$ entities, each taken from entity sets

$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

where $(e_1, e_2, \dots, e_n)$ is a relationship

- Example: $(\text{Hayes}, \text{A-102}) \in \text{depositor}$

Relationship Sets Borrower

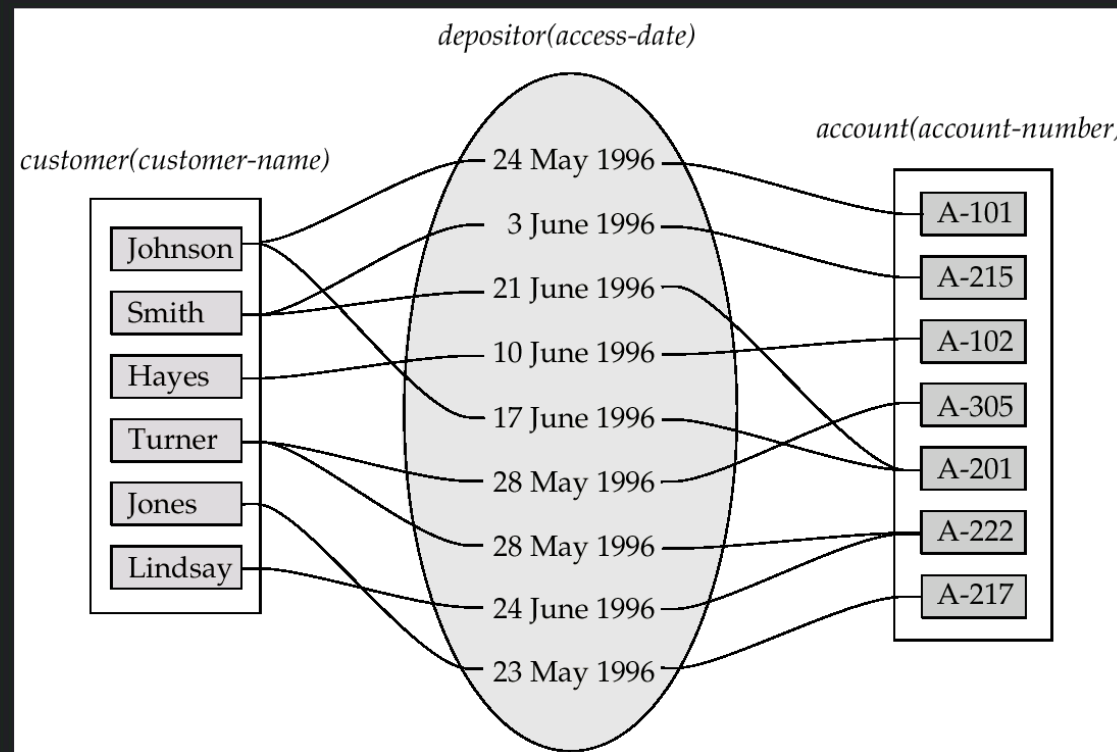


Relationship Sets

- The association between entity sets is referred to as **participation**.
- A **relationship instance** in an E-R schema represents an association between the named entities that is being modeled.
- An entity plays in a relationship is called that entity's **role**.
- The entity sets of a relationship set are not distinct; that is, the same entity set participates in a relationship set more than once, in different roles, sometimes called a **recursive relationship set**, explicit role names are to be specified how an entity participates in a relationship.

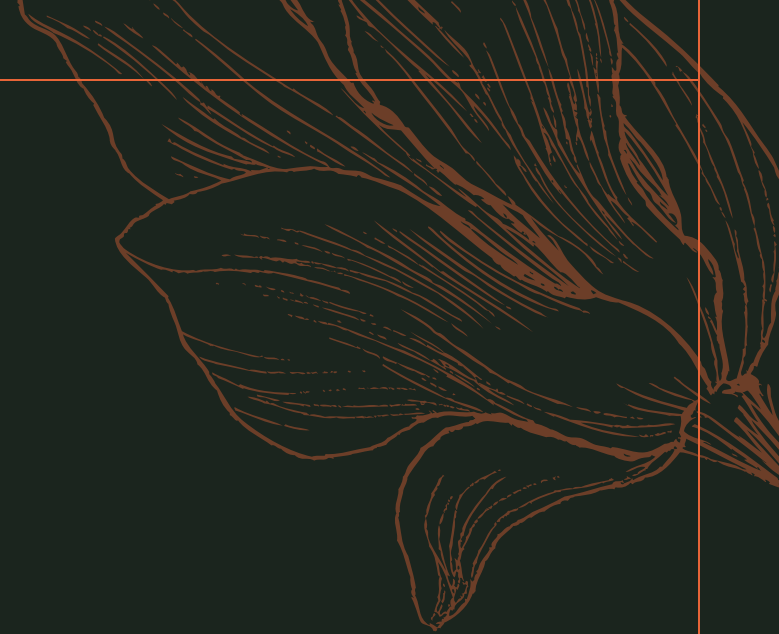
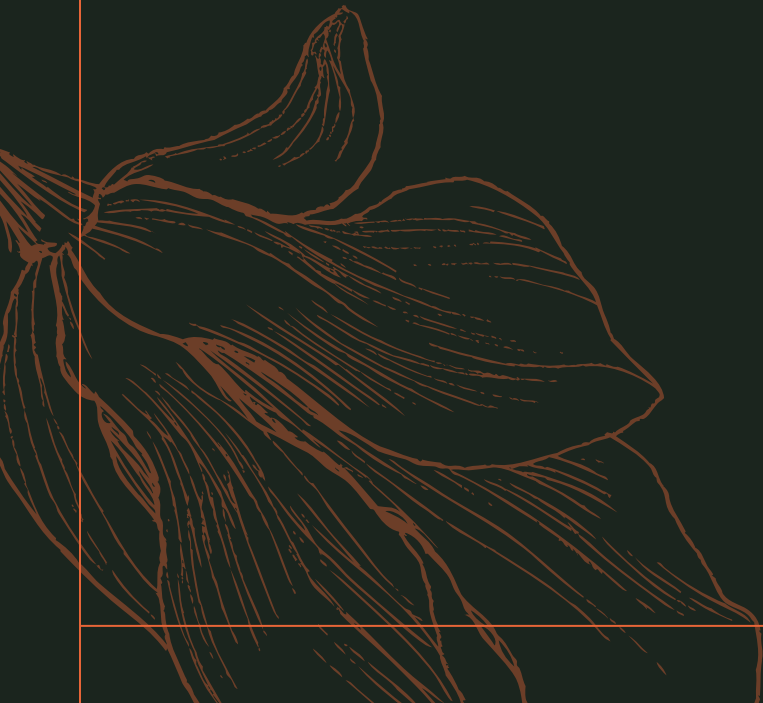
Relationship Sets Borrower

- An **attribute** can also be property of a relationship set, called as **Descriptive Attribute**
- For instance, the depositor relationship set between entity sets customer and account may have the attribute access-date



Constraints

Entity-Relationship Modeling



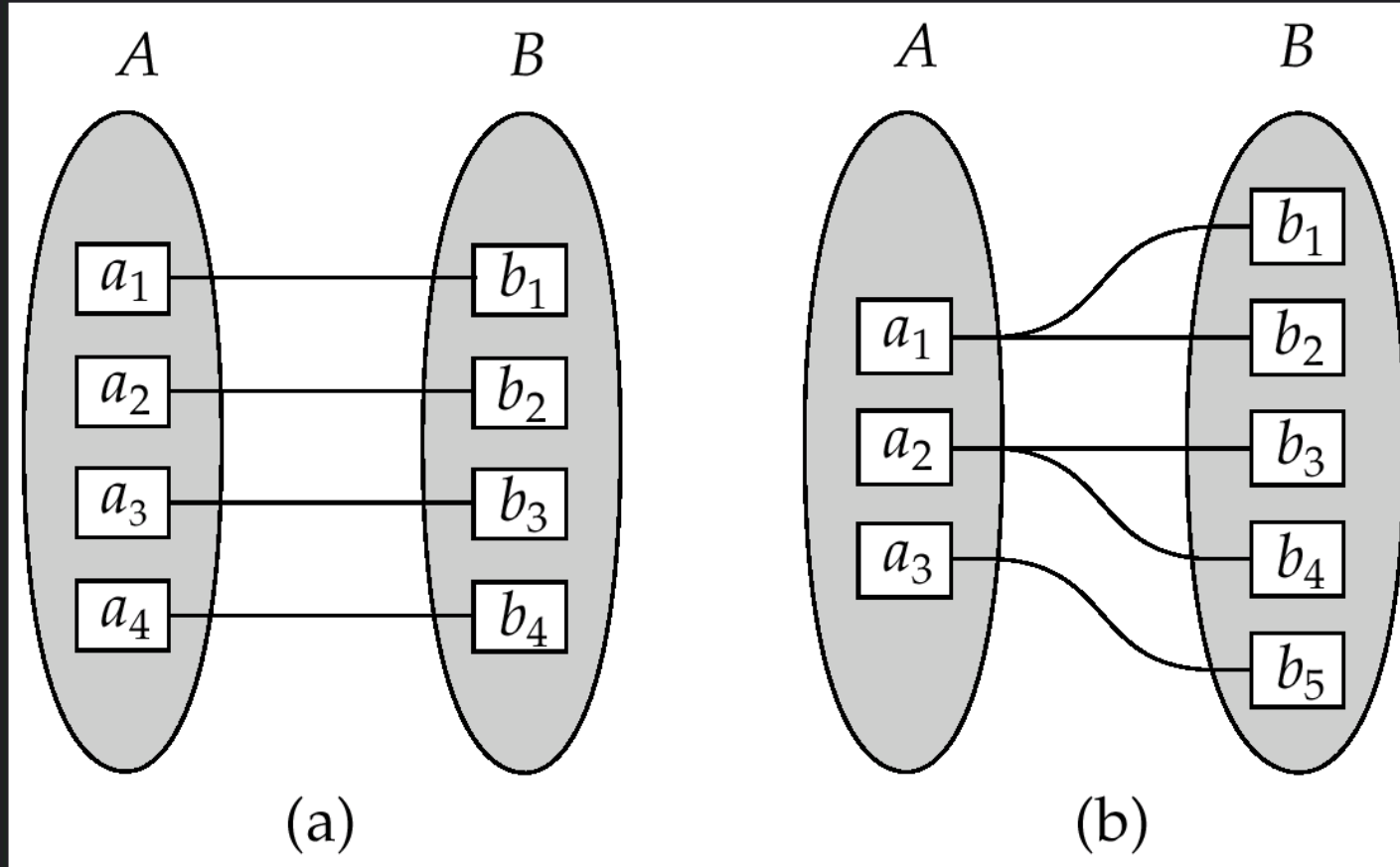
Degree of a Relationship Sets

- Refers to number of entity sets that participate in a relationship set.
- Relationship sets that involve two entity sets are **binary** (or degree two). Generally, most relationship sets in a database system are binary.
- Relationship sets may involve more than two entity sets.
Suppose employees of a bank may have jobs (responsibilities) at multiple branches, with different jobs at different branches. Then there is a ternary relationship set between entity sets employee, job and branch
- Relationships between more than two entity sets are rare. Most relationships are binary.

Mapping Cardinalities

- For a binary relationship set R between entity sets A and B , the mapping cardinality must be one of the following:
- **One to one.** An entity in A is associated with at most one entity in B , and an entity in B is associated with at most one entity in A . (See Figure a.)
- **One to many.** An entity in A is associated with any number (zero or more) of entities in B . An entity in B , however, can be associated with at most one entity in A . (See Figure b.)

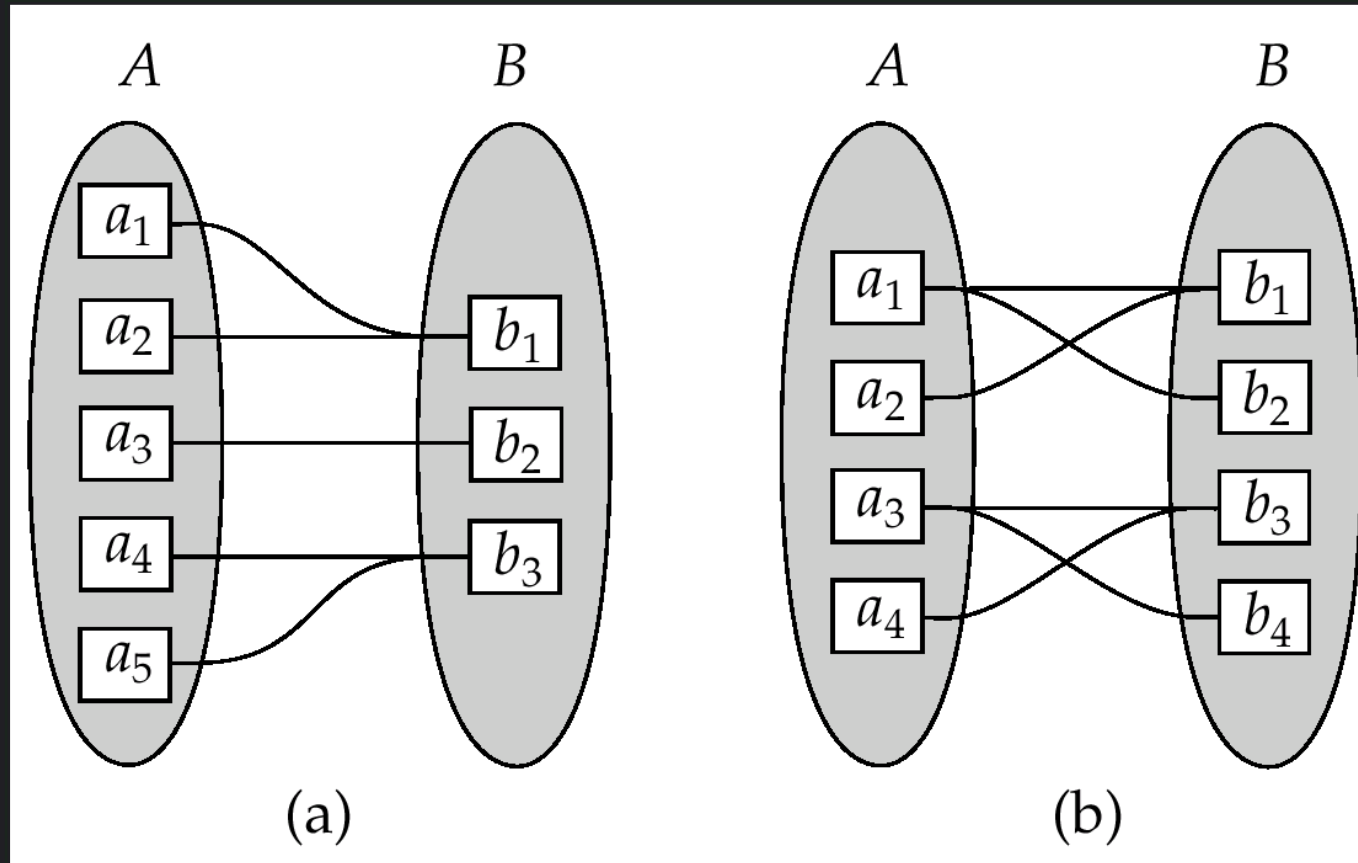
Mapping Cardinalities



Mapping Cardinalities

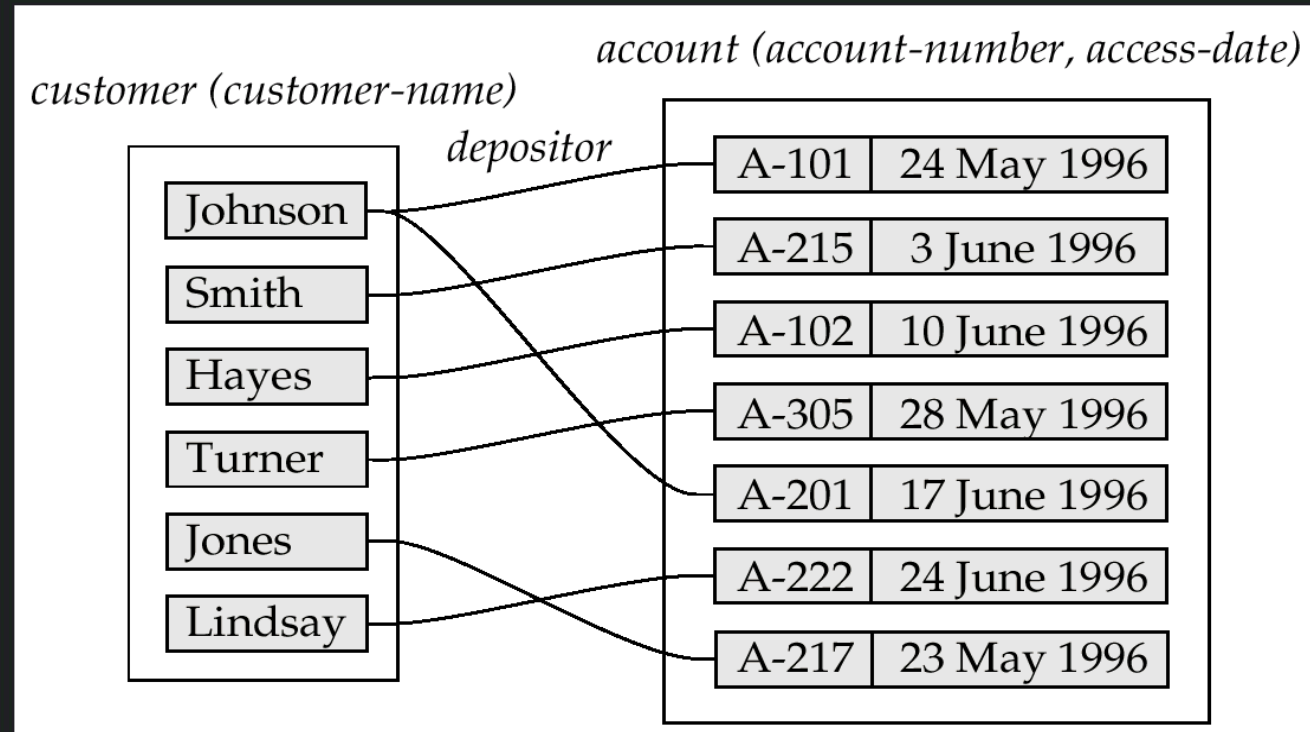
- **Many to one.** An entity in A is associated with at most one entity in B. An entity in B, however, can be associated with any number (zero or more) of entities in A. (See Figure a.)
- **Many to many.** An entity in A is associated with any number (zero or more) of entities in B, and an entity in B is associated with any number (zero or more) of entities in A. (See Figure b.)

Mapping Cardinalities



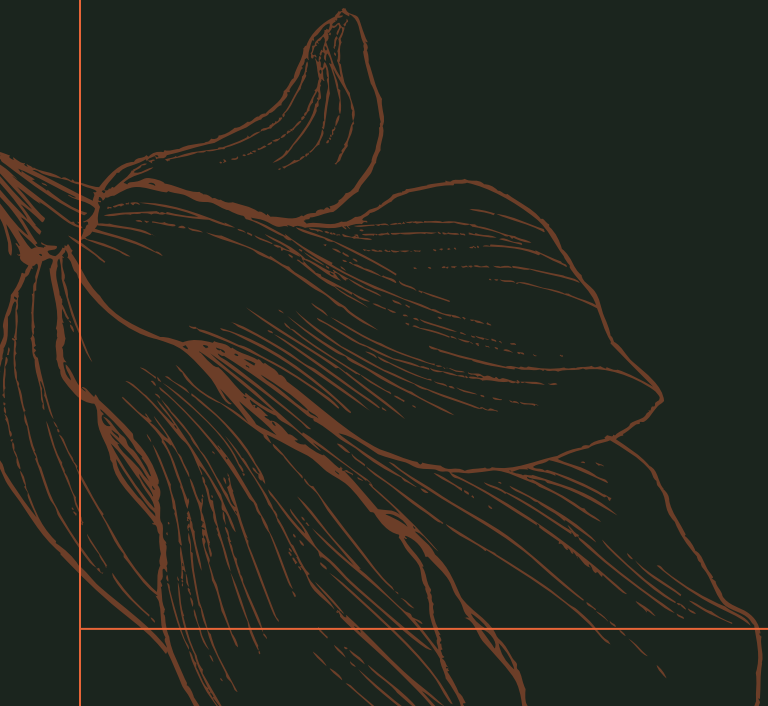
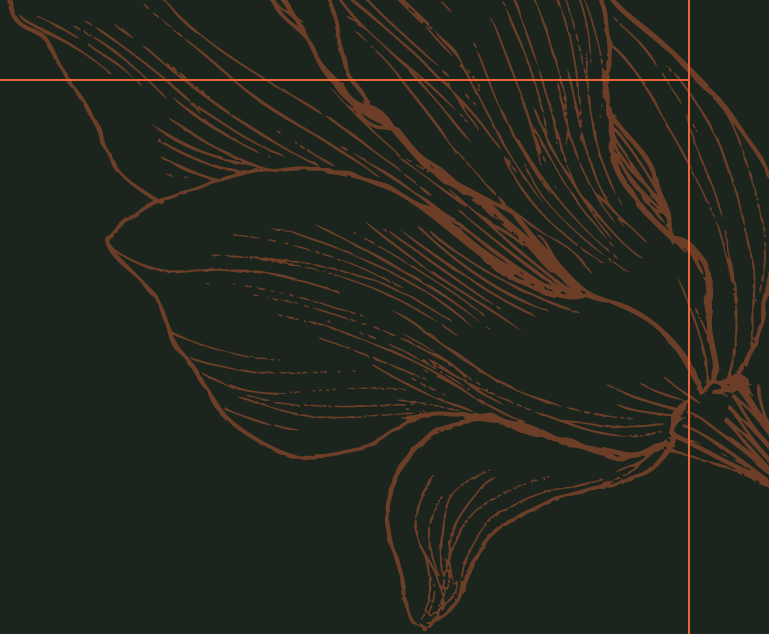
Mapping Cardinalities affect ER Design

- An Can make access-date an attribute of account, instead of a relationship attribute, if each account can have only one customer
- i.e., the relationship from account to customer is many to one, or equivalently, customer to account is one to many



Participation Constraints

- The participation of an entity set E in a relationship set R is said to be **total** if every entity in E participates in at least one relationship in R .
- If only some entities in E participate in relationships in R , the participation of entity set E in relationship R is said to be **partial**.
- The participation of loan in the relationship set borrower is **total**.
- In contrast, an individual can be a bank customer whether or not she has a loan with the bank.
- Only some of the customer entities are related to the loan entity set through the borrower relationship, and the participation of customer in the borrower relationship set is therefore partial.



Keys

Entity-Relationship Modeling

Keys

- To specify how entities within a given entity set are distinguished.
- Conceptually, individual entities are distinct; from a database perspective, however, the difference among them must be expressed in terms of their attributes.
- Therefore, the values of the attribute values of an entity must be such that they can uniquely identify the entity.
- In other words, no two entities in an entity set are allowed to have exactly the same value for all attributes.
- A **key** allows us to identify a set of attributes that suffice to distinguish entities from each other.
- Keys also help uniquely identify relationships, and thus distinguish relationships from each other.

Keys – Entity Set

- A **superkey** is a set of one or more attributes that, taken collectively, allow us to identify uniquely an entity in the entity set.
- For example, the customer-id attribute of the entity set customer is sufficient to distinguish one customer entity from another.
- If K is a superkey, then so is any superset of K . We are often interested in superkeys for which no proper subset is a superkey. Such minimal superkeys are called **candidate keys**.
- **Primary key** to denote a candidate key that is chosen by the database designer as the principal means of identifying entities within an entity set.

Keys – Relationship Set

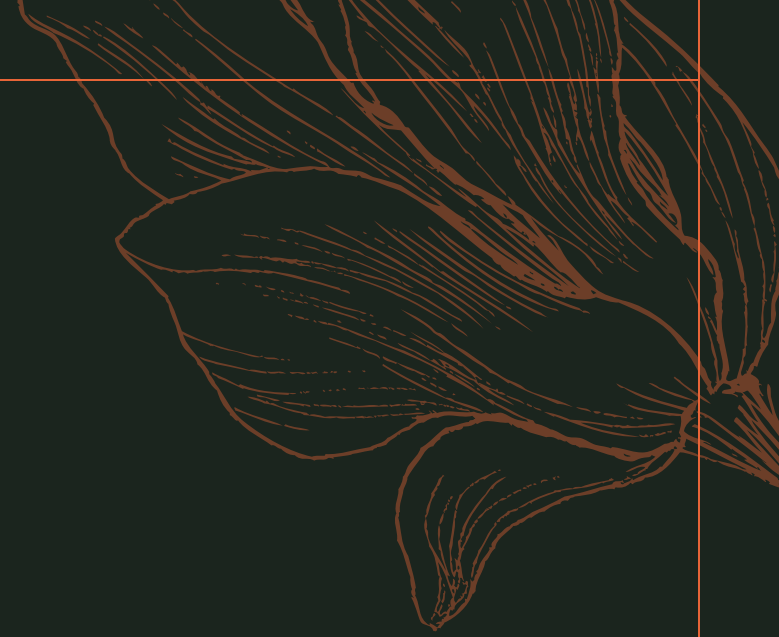
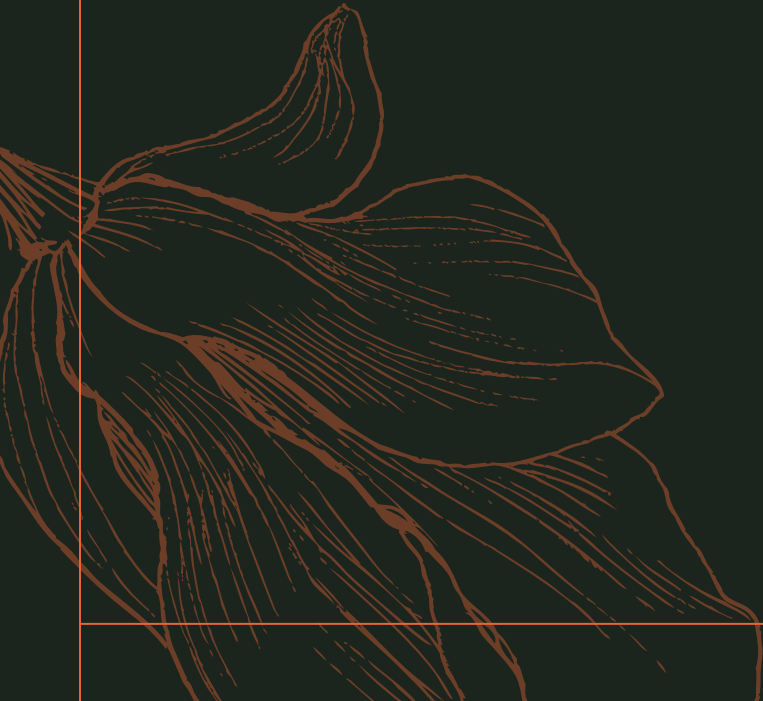
- Let R be a relationship set involving entity sets $E_1, E_2, \dots, E_n$. Let $\text{primary-key}(E_i)$ denote the set of attributes that forms the primary key for entity set E_i .
- The attribute names of all primary keys are unique, and each entity set participates only once in the relationship.
- If the relationship set R has no attributes associated with it, then the set of attributes
 $\text{primary-key}(E_1) \cup \text{primary-key}(E_2) \cup \dots \cup \text{primary-key}(E_n)$

Keys – Relationship Set

- Let R be a relationship set involving entity sets $E_1, E_2, \dots, E_n$. Let $\text{primary-key}(E_i)$ denote the set of attributes that forms the primary key for entity set E_i .
- The attribute names of all primary keys are unique, and each entity set participates only once in the relationship.
- If the relationship set R has no attributes associated with it, then the set of attributes
 $\text{primary-key}(E_1) \cup \text{primary-key}(E_2) \cup \dots \cup \text{primary-key}(E_n)$

Design Issues

Entity-Relationship Modeling

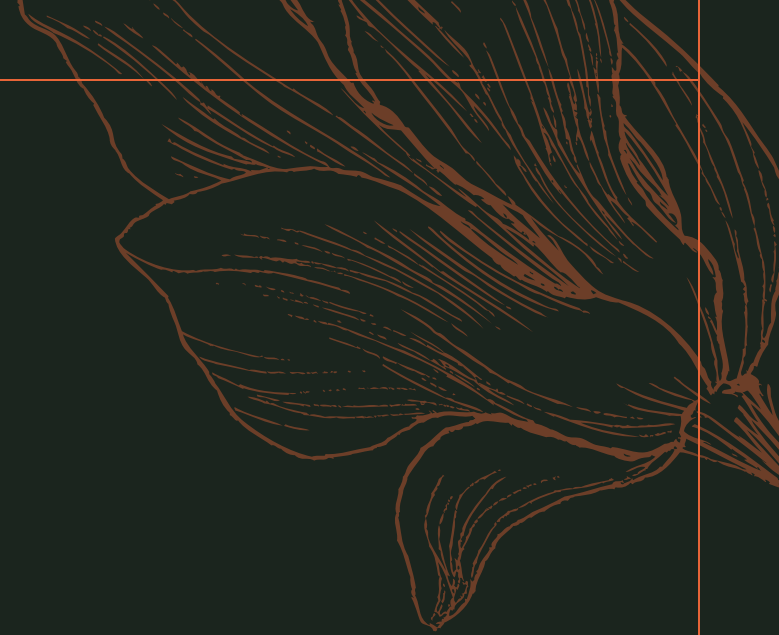
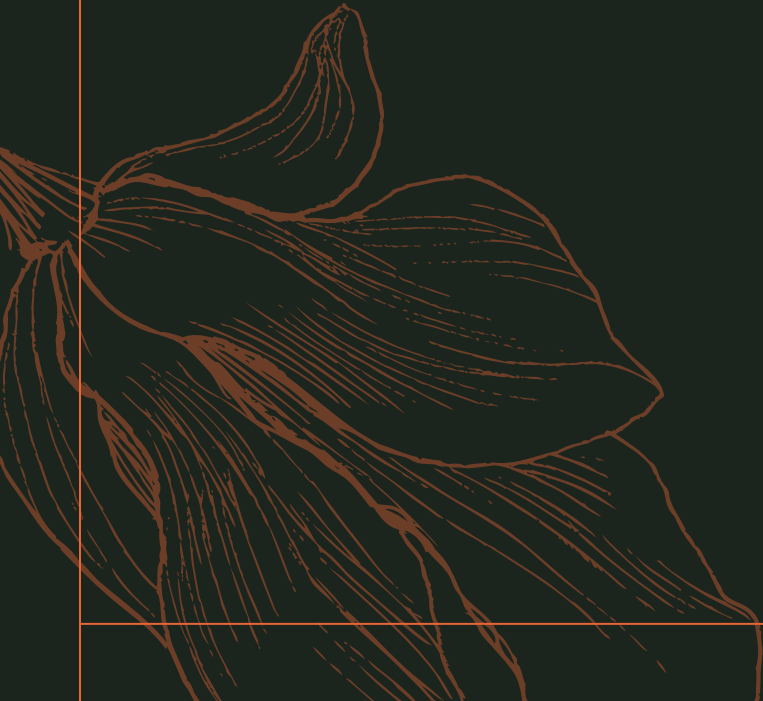


Design Issues

- **Use of entity sets vs. attributes**
 - Choice mainly depends on the structure of the enterprise being modeled, and on the semantics associated with the attribute in question.
- **Use of entity sets vs. relationship sets**
 - Possible guideline is to designate a relationship set to describe an action that occurs between entities
- **Binary versus n-ary relationship sets**
 - Although it is possible to replace any nonbinary (n-ary, for $n > 2$) relationship set by a number of distinct binary relationship sets, a n-ary relationship set shows more clearly that several entities participate in a single relationship.
- **Placement of relationship attributes**

E-R Diagrams

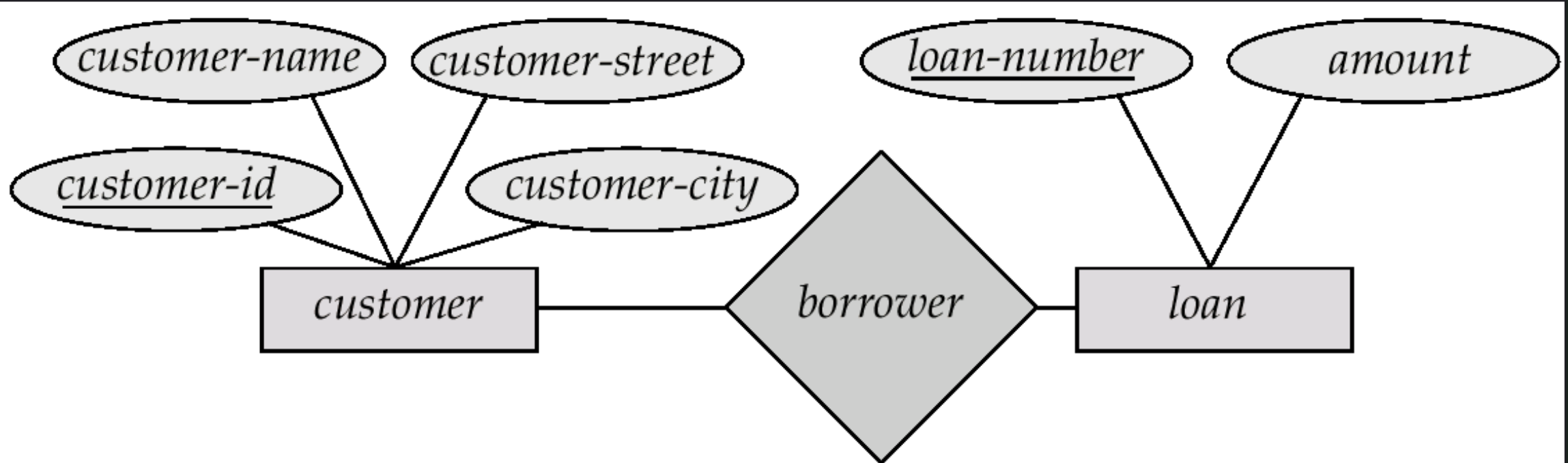
Entity-Relationship Modeling



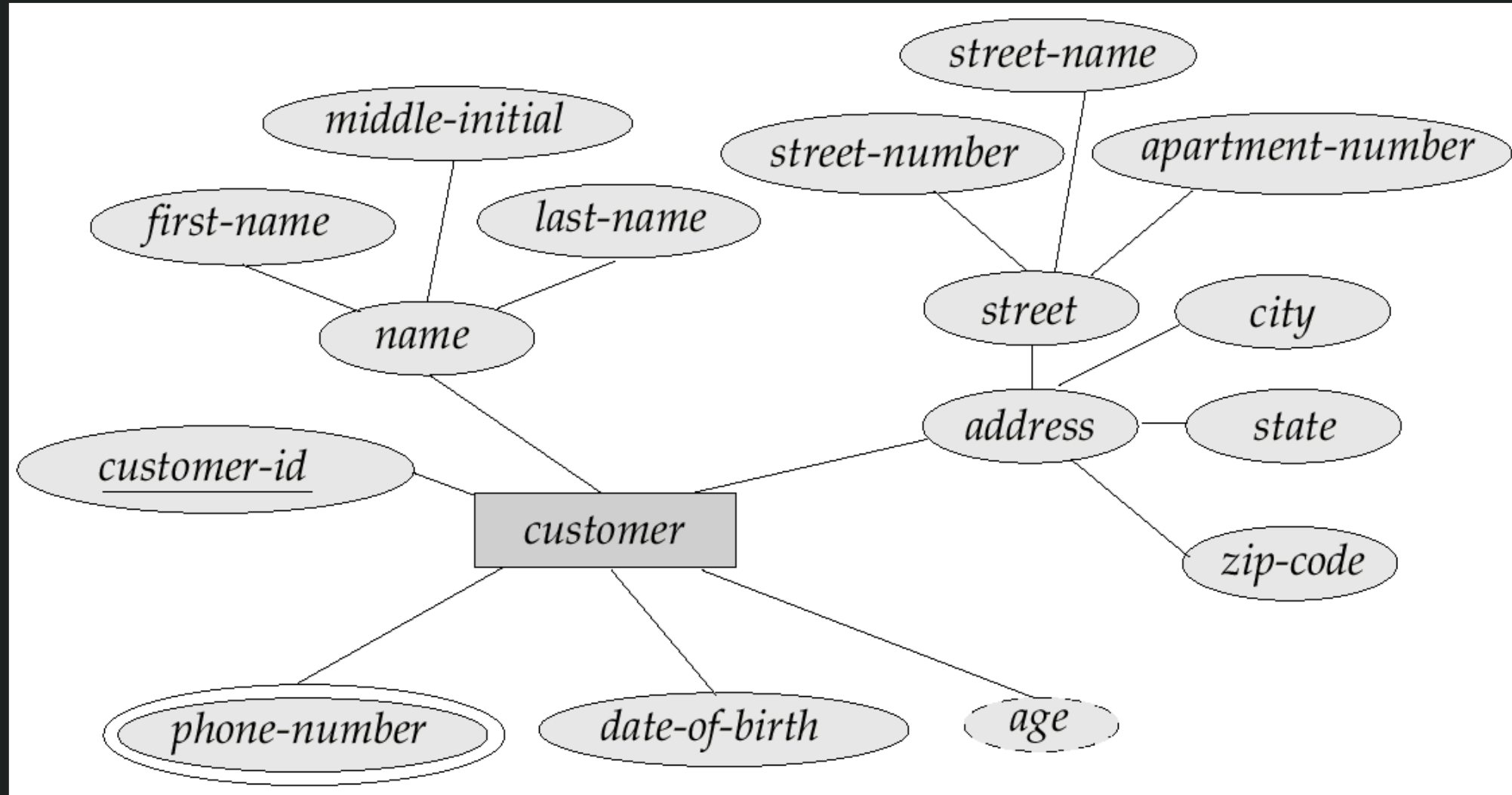
E-R Diagrams

- **Rectangles** represent entity sets.
- **Diamonds** represent relationship sets.
- **Lines** link attributes to entity sets and entity sets to relationship sets.
- **Ellipses** represent attributes
 - **Double** ellipses represent multivalued attributes.
 - **Dashed** ellipses denote derived attributes.
- **Underline** indicates primary key attributes
- **Double lines**, which indicate total participation of an entity in a relationship set
- **Double rectangles**, which represent weak entity sets

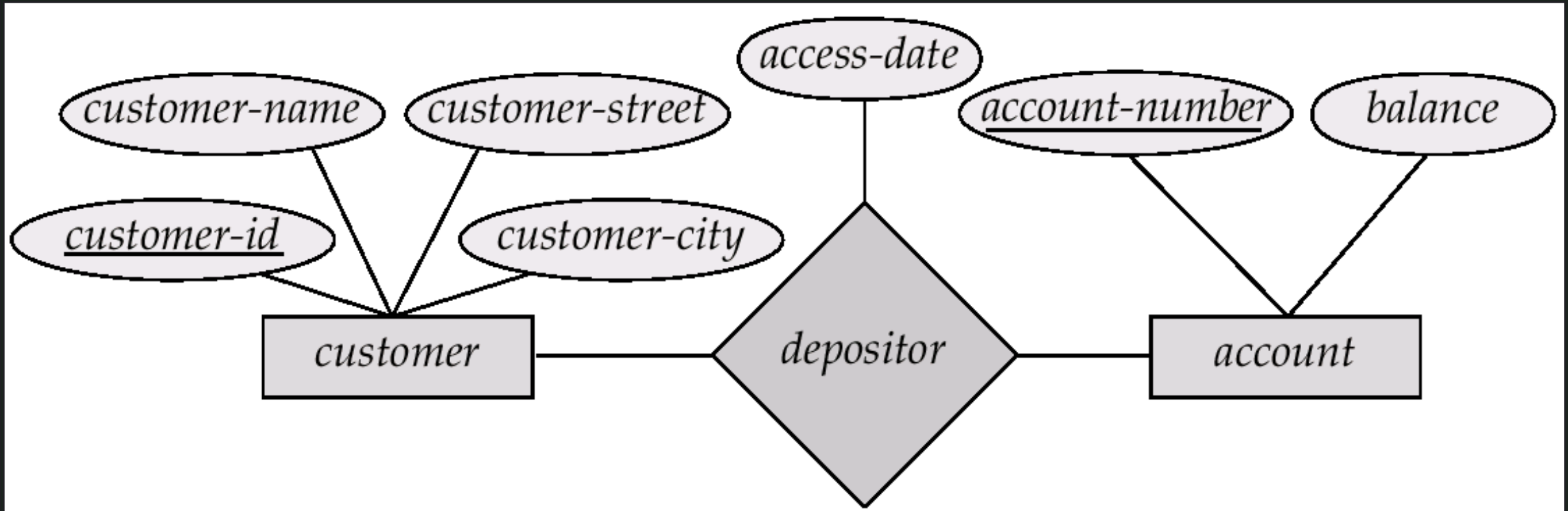
Degree of a Relationship Sets



Identify composite, Multivalued and derived attribute

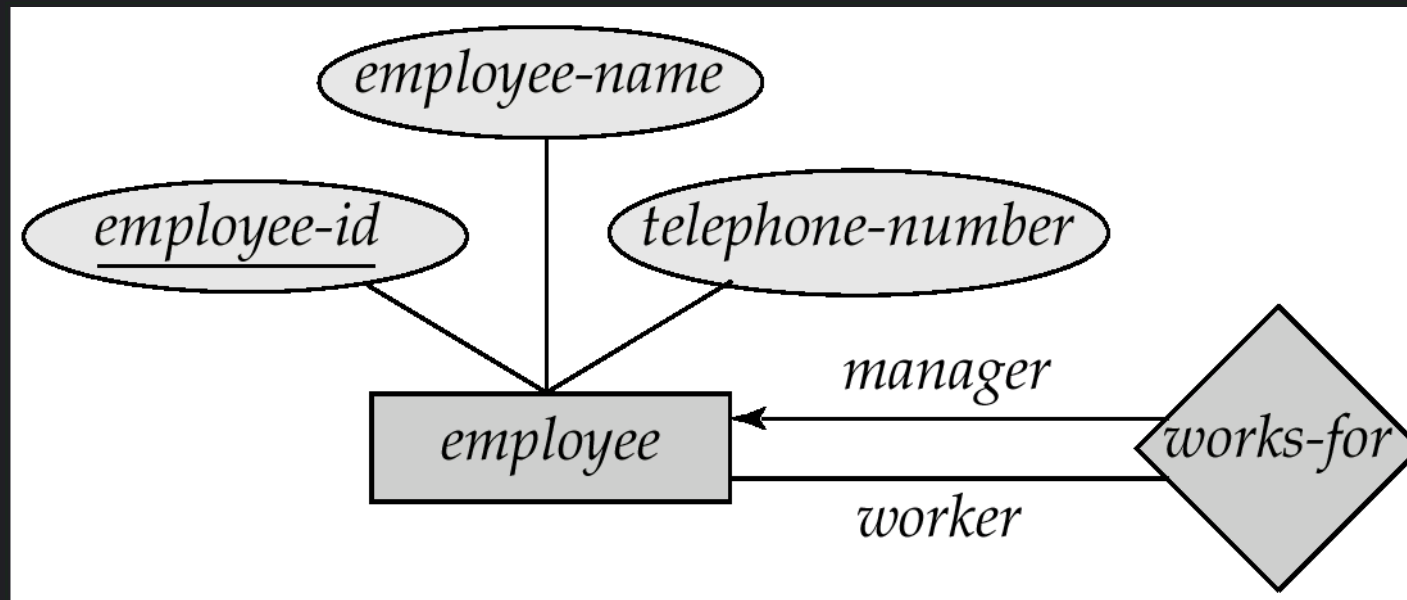


Relationship Sets with Attributes



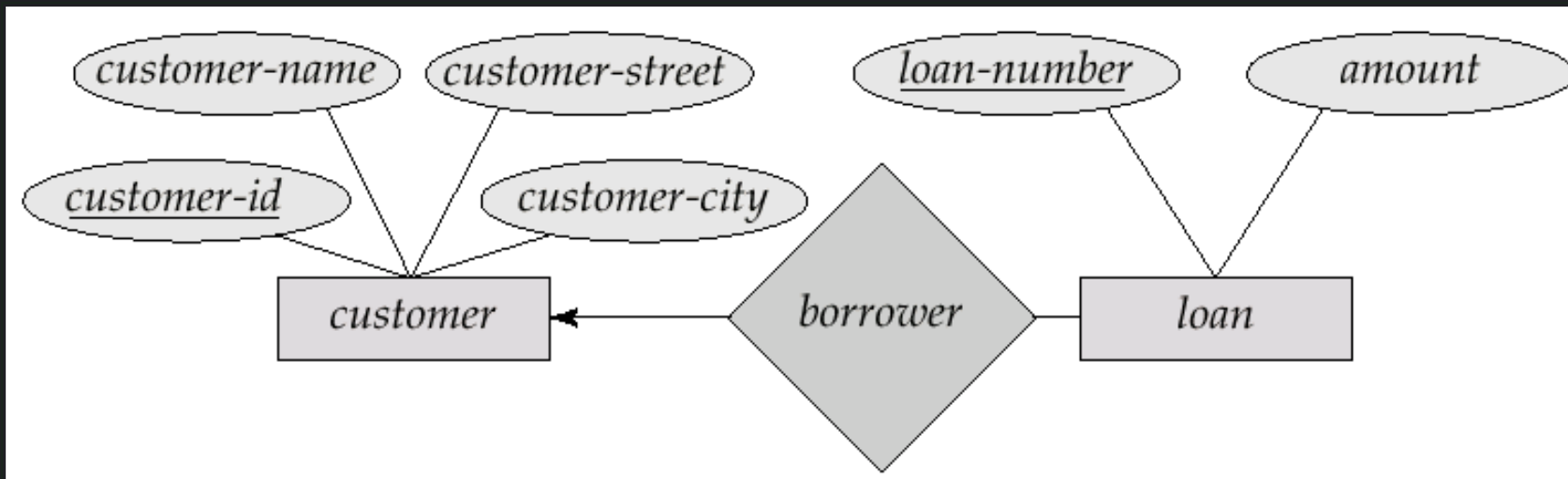
Roles

- Entity sets of a relationship need not be distinct
- The labels “manager” and “worker” are called **roles**; they specify how employee entities interact via the works-for relationship set.
- Roles are indicated in E-R diagrams by labeling the lines that connect diamonds to rectangles.
- Role labels are optional, and are used to clarify semantics of the relationship



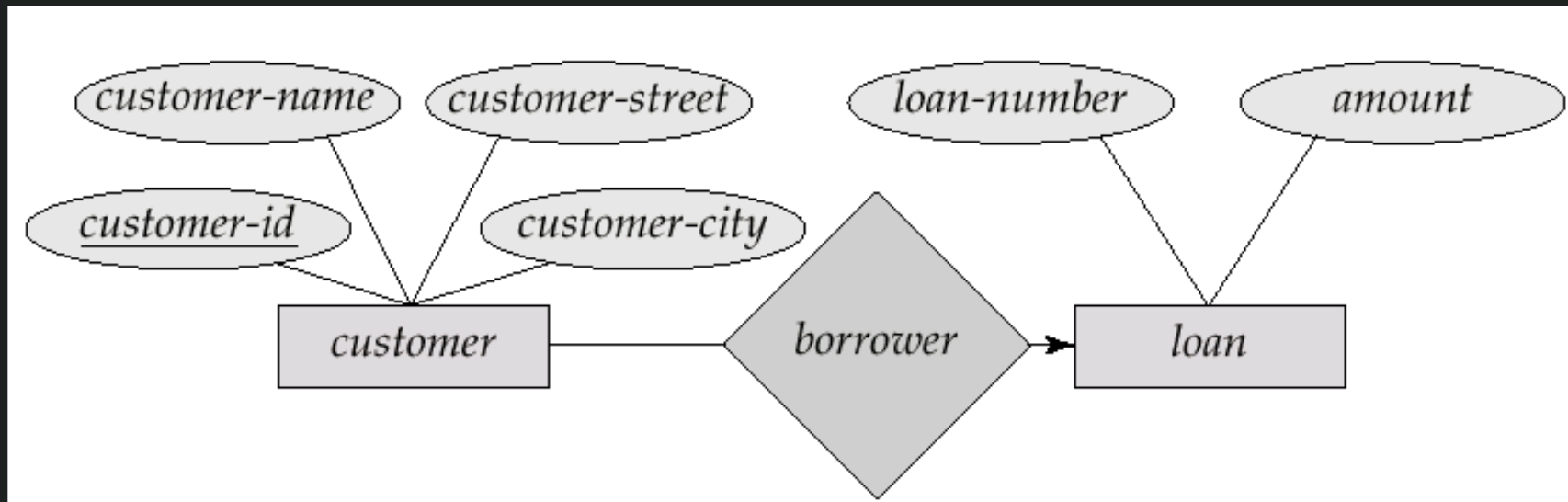
One to Many Relationship

- In the one-to-many relationship a loan is associated with at most one customer via borrower, a customer is associated with several (including 0) loans via borrower



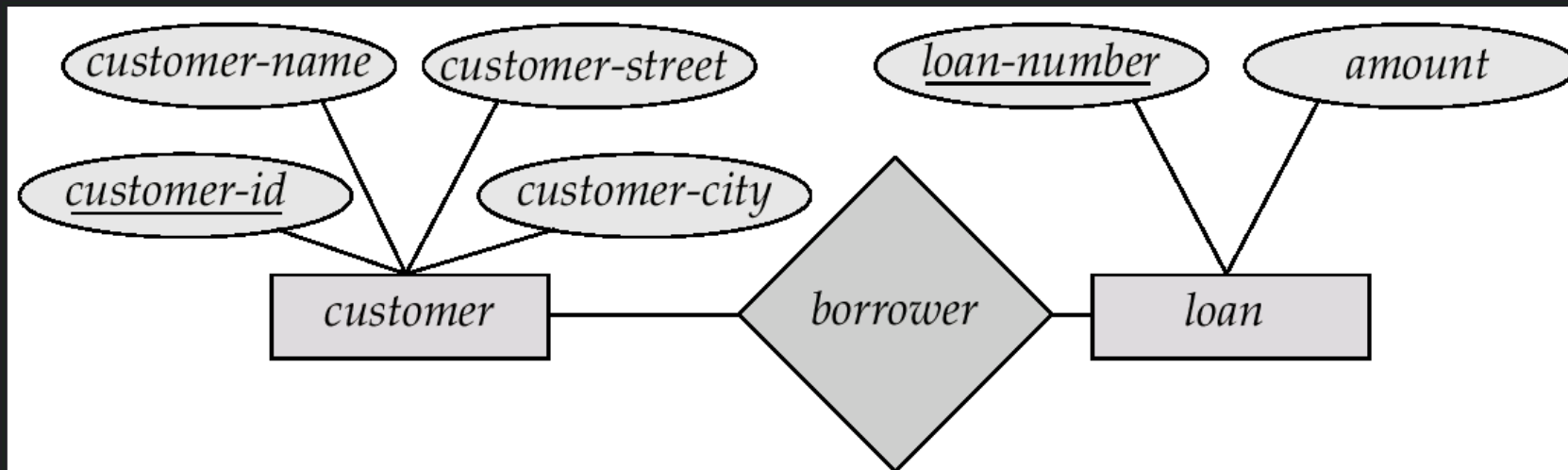
Many to One Relationship

- In a many-to-one relationship a loan is associated with several (including 0) customers via borrower, a customer is associated with at most one loan via borrower



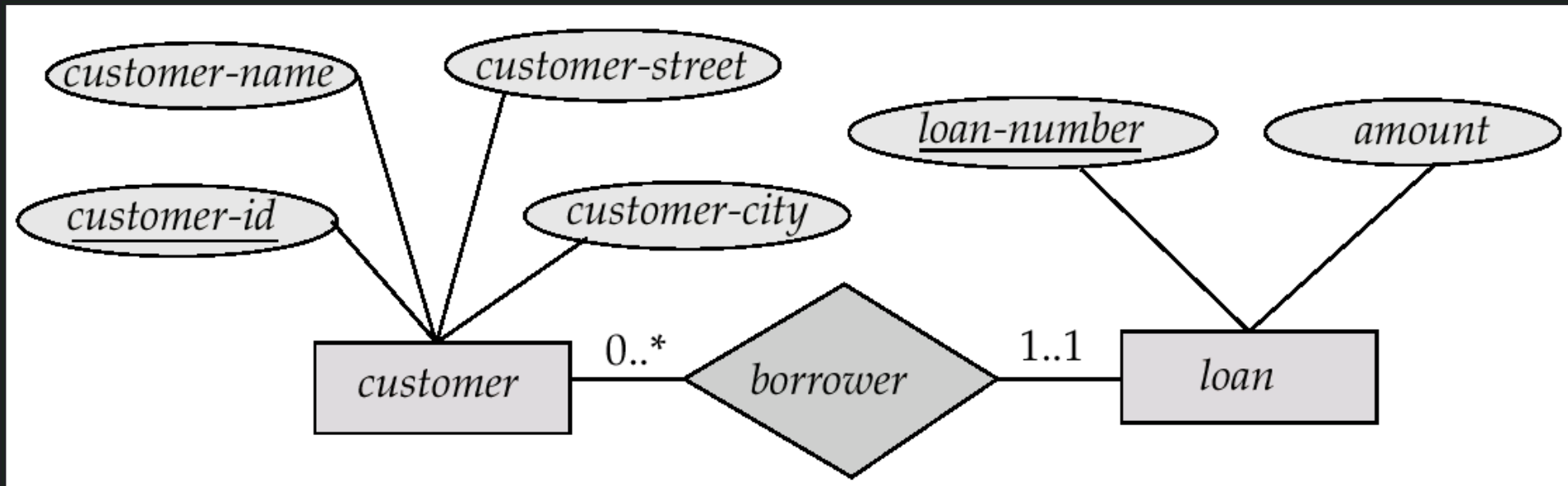
Many to Many Relationship

- A customer is associated with several (possibly 0) loans via borrower
- A loan is associated with several (possibly 0) customers via borrower



Alternate Notation for Cardinality Limits

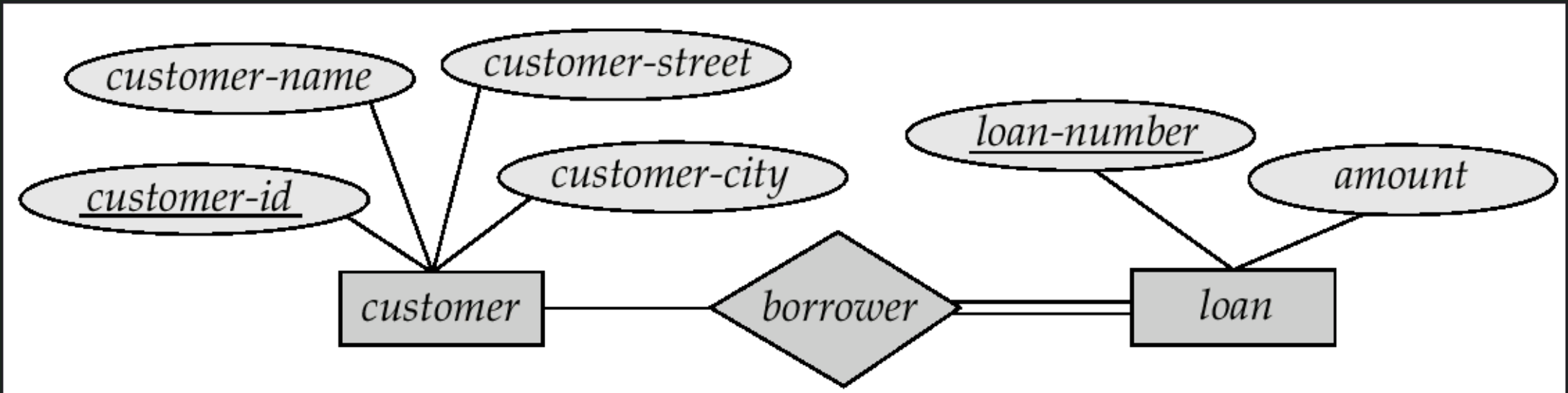
- Cardinality limits can also express participation constraints



Participation of an Entity Set in a Relationship set

- **Total participation** (indicated by double line): every entity in the entity set participates in at least one relationship in the relationship set
- E.g. participation of loan in borrower is total
- every loan must have a customer associated to it via borrower
- **Partial participation**: some entities may not participate in any relationship in the relationship set
- E.g. participation of customer in borrower is partial

Participation of an Entity Set in a Relationship set



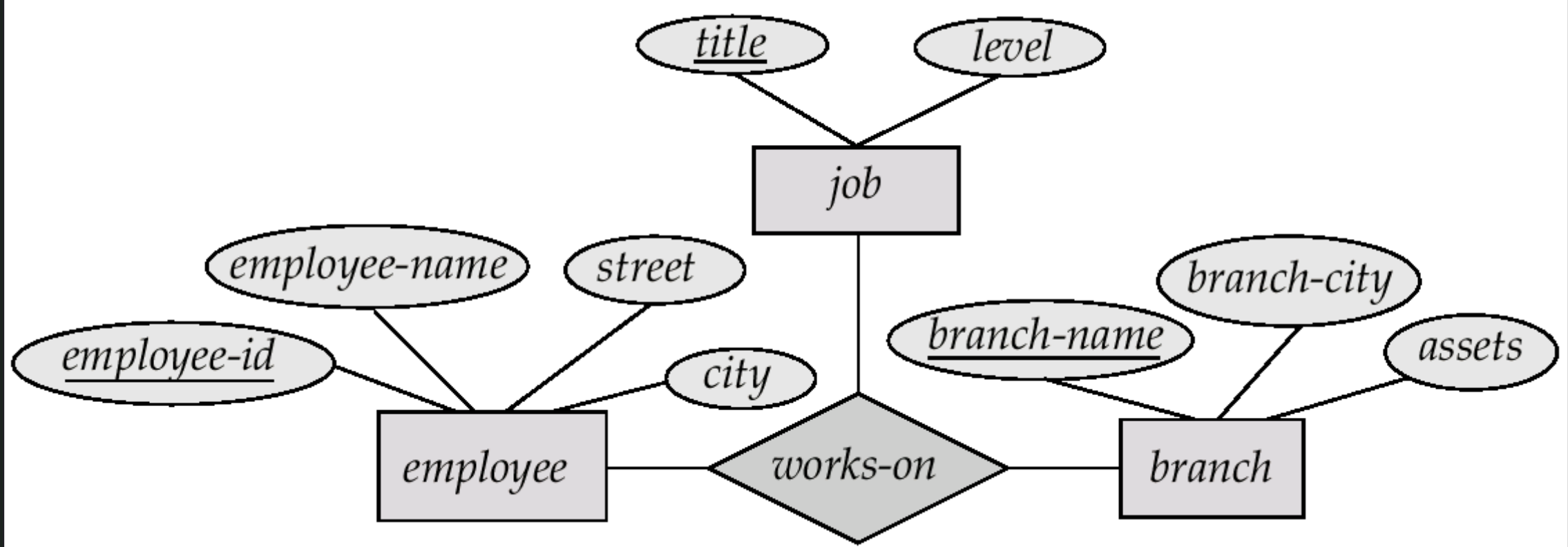
Keys

- A **super key** of an entity set is a set of one or more attributes whose values uniquely determine each entity.
- A **candidate key** of an entity set is a minimal super key
- Customer-id is candidate key of customer
- account-number is candidate key of account
- Although several candidate keys may exist, one of the candidate keys is selected to be the primary key.

Keys for Relationship Sets

- The combination of primary keys of the participating entity sets forms a super key of a relationship set.
 - (customer-id, account-number) is the super key of depositor
 - NOTE: this means a pair of entity sets can have at most one relationship in a particular relationship set.
 - E.g. if we wish to track all access-dates to each account by each customer, we cannot assume a relationship for each access. We can use a multivalued attribute though
- Must consider the mapping cardinality of the relationship set when deciding what are the candidate keys
- Need to consider semantics of relationship set in selecting the primary key in case of more than one candidate key

E-R Diagram with a Ternary Relationship



E-R Diagram with a Ternary Relationship

- We allow at most **one arrow out of a ternary (or greater degree) relationship to indicate a cardinality constraint**
- E.g. an arrow from works-on to job indicates each employee works on at most one job at any branch.
- If there is more than one arrow, there are two ways of defining the meaning.
 - E.g a ternary relationship R between A, B and C with arrows to B and C could mean
 - each A entity is associated with a unique entity from B and C or
 - each pair of entities from (A, B) is associated with a unique C entity, and each pair (A, C) is associated with a unique B
 - Each alternative has been used in different formalisms
 - To avoid confusion we outlaw more than one arrow

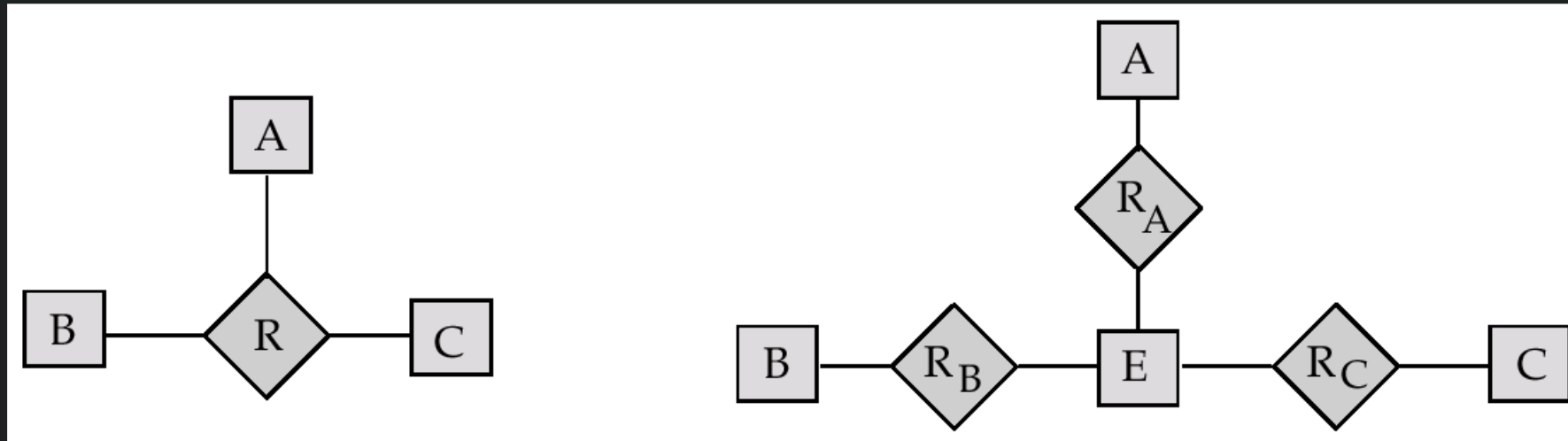
Converting Non-Binary to Binary Relationships

- Some relationships that appear to be non-binary may be better represented using binary relationships
 - E.g. A ternary relationship parents, relating a child to his/her father and mother, is best replaced by two binary relationships, father and mother
 - Using two binary relationships allows partial information (e.g. only mother being know)
 - But there are some relationships that are naturally non-binary
 - E.g. works-on

Converting Non-Binary to Binary Relationships

- In general, any non-binary relationship can be represented using binary relationships by creating an artificial entity set.
 - Replace R between entity sets A, B and C by an entity set E, and three relationship sets:
 1. RA, relating E and A
 2. RB, relating E and B
 3. RC, relating E and C
 - Create a special identifying attribute for E
 - Add any attributes of R to E
 - For each relationship (a_i, b_i, c_i) in R, create
 1. a new entity e_i in the entity set E
 2. add (e_i, a_i) to RA
 3. add (e_i, b_i) to RB
 4. add (e_i, c_i) to RC

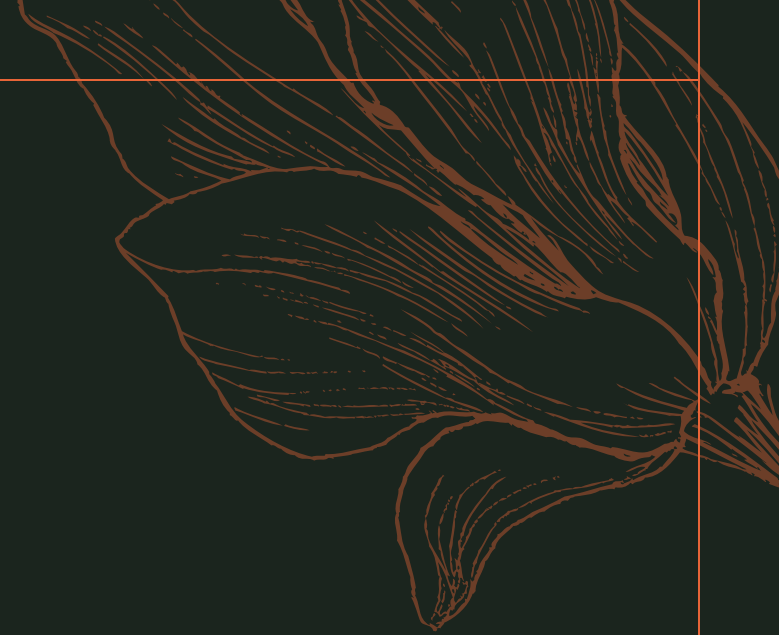
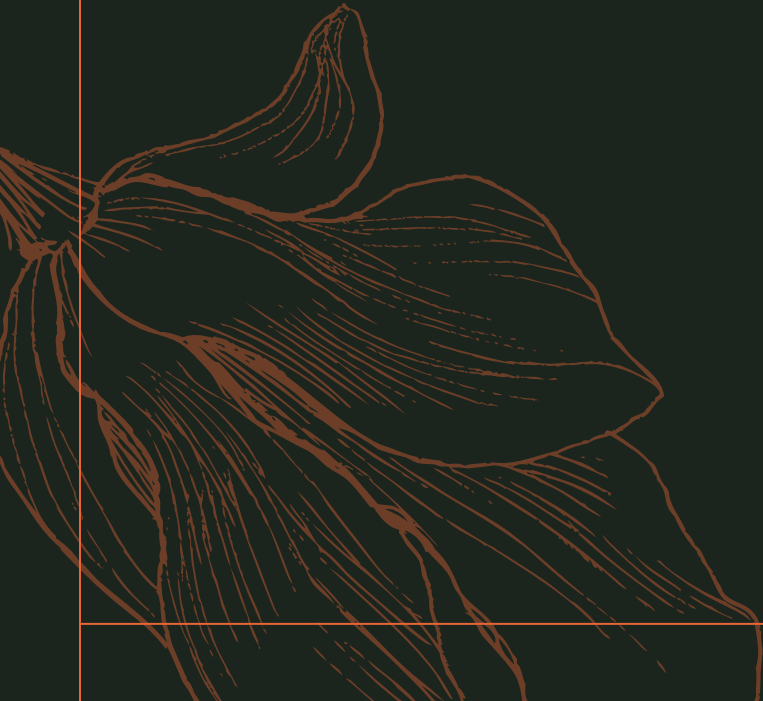
Converting Non-Binary to Binary Relationships



- Also need to translate constraints
 - Translating all constraints may not be possible
 - There may be instances in the translated schema that cannot correspond to any instance of R
 - Exercise: add constraints to the relationships R_A , R_B and R_C to ensure that a newly created entity corresponds to exactly one entity in each of entity sets A , B and C
 - We can avoid creating an identifying attribute by making E a weak entity set (described shortly) identified by the three relationship sets

Weak Entity Sets

Entity-Relationship Modeling



Weak Entity Set

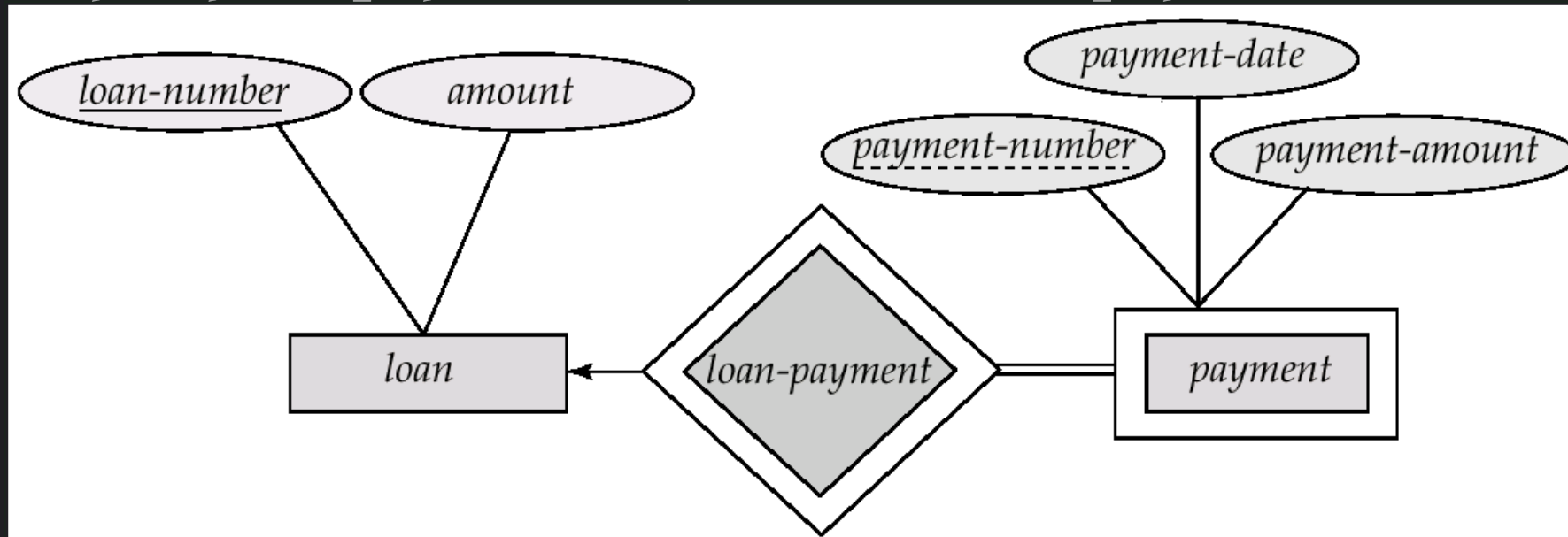
- An entity set that does not have a primary key is referred to as a **weak entity set**.
- The existence of a **weak entity set depends on the existence of a identifying entity set**
 - it must relate to the identifying entity set via a total, one-to-many relationship set from the identifying to the weak entity set
 - **Identifying relationship** depicted using a double diamond
- The **discriminator** (or partial key) of a weak entity set is the **set of attributes that distinguishes among all the entities of a weak entity set**.
- The **primary key** of a weak entity set is formed by the **primary key of the strong entity set** on which the weak entity set is existence dependent, plus the weak entity set's discriminator.

Weak Entity Set

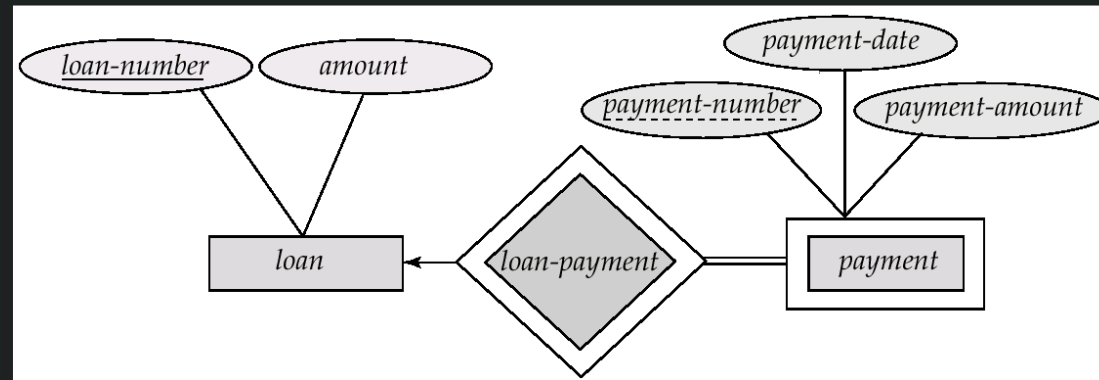
- An entity set that does not have a primary key is referred to as a **weak entity set**.
- The existence of a **weak entity set depends on the existence of a identifying entity set**
 - It must relate to the identifying entity set via a total, one-to-many relationship set from the identifying to the weak entity set
 - **Identifying relationship** depicted using a double diamond
- The **discriminator** (or partial key) of a weak entity set is the **set of attributes that distinguishes among all the entities of a weak entity set**.
- The **primary key of a weak entity set is formed by the primary key of the strong entity set on which the weak entity set is existence dependent, plus the weak entity set's discriminator**.

Weak Entity Set

- We depict a **weak entity set** by double rectangles.
- We **underline** the discriminator of a weak entity set with a dashed line.
- **payment-number** – discriminator of the payment entity set
- Primary key for payment – (loan-number, payment-number)



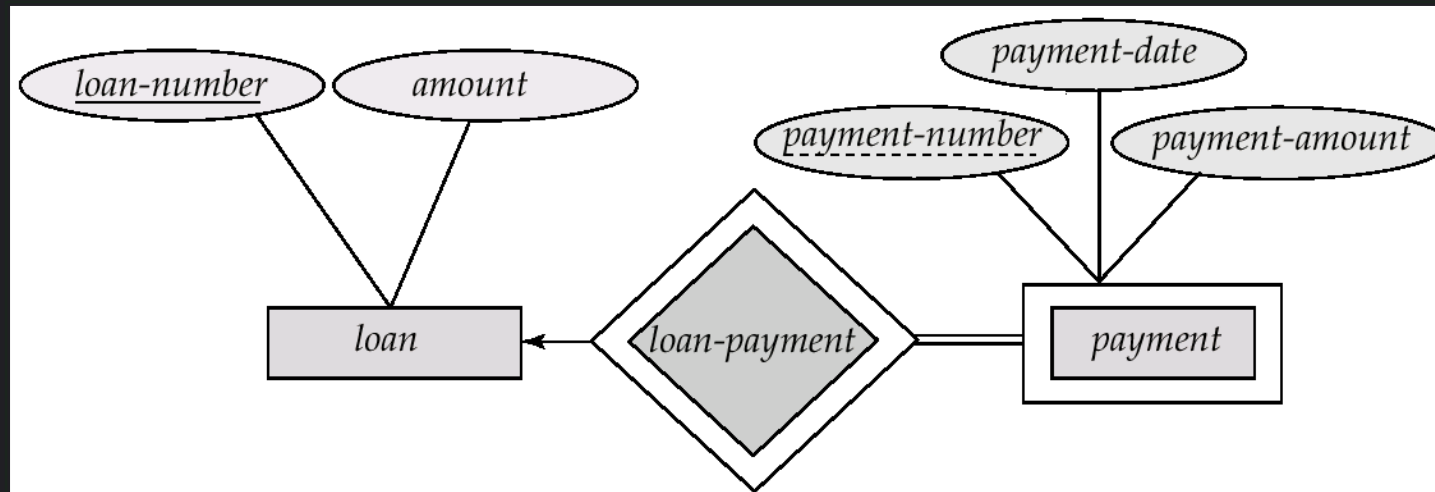
Weak Entity Set



<i>loan-number</i>	<i>payment-number</i>	<i>payment-date</i>	<i>payment-amount</i>
L-11	53	7 June 2001	125
L-14	69	28 May 2001	500
L-15	22	23 May 2001	300
L-16	58	18 June 2001	135
L-17	5	10 May 2001	50
L-17	6	7 June 2001	50
L-17	7	17 June 2001	100
L-23	11	17 May 2001	75
L-93	103	3 June 2001	900
L-93	104	13 June 2001	200

Weak Entity Set

- Primary key of the strong entity set is not explicitly stored with the weak entity set, since it is implicit in the identifying relationship.
- If loan-number were explicitly stored, payment could be made a strong entity, but then the relationship between payment and loan would be duplicated by an implicit relationship defined by the attribute loan-number common to payment and loan

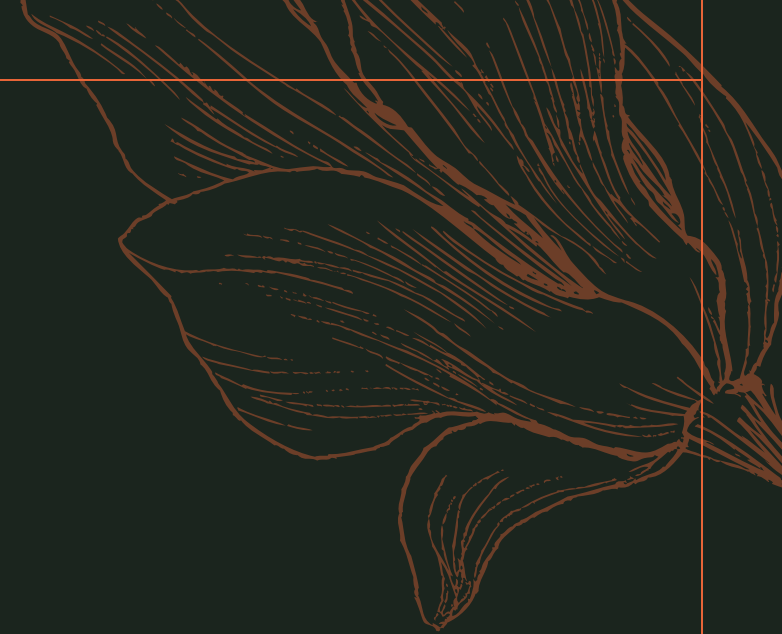
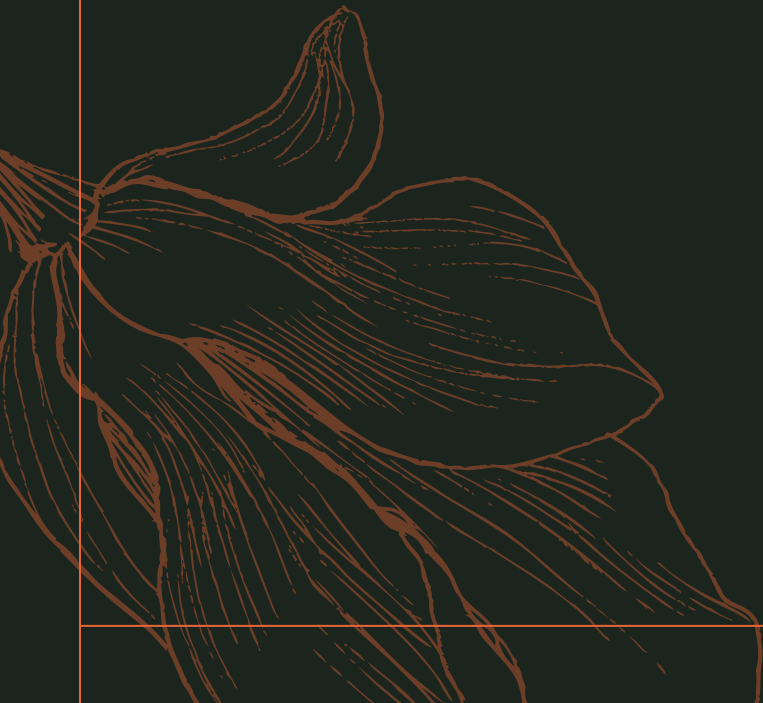


Weak Entity Set

- In a university, a **course** is a strong entity and a **course-offering** can be modeled as a weak entity
- The discriminator of course-offering would be semester (including year) and section-number (if there is more than one section)
- If we model course-offering as a strong entity we would model course-number as an attribute.
- Then the relationship with course would be implicit in the course-number attribute

Extended E-R Feature

Entity-Relationship Modeling

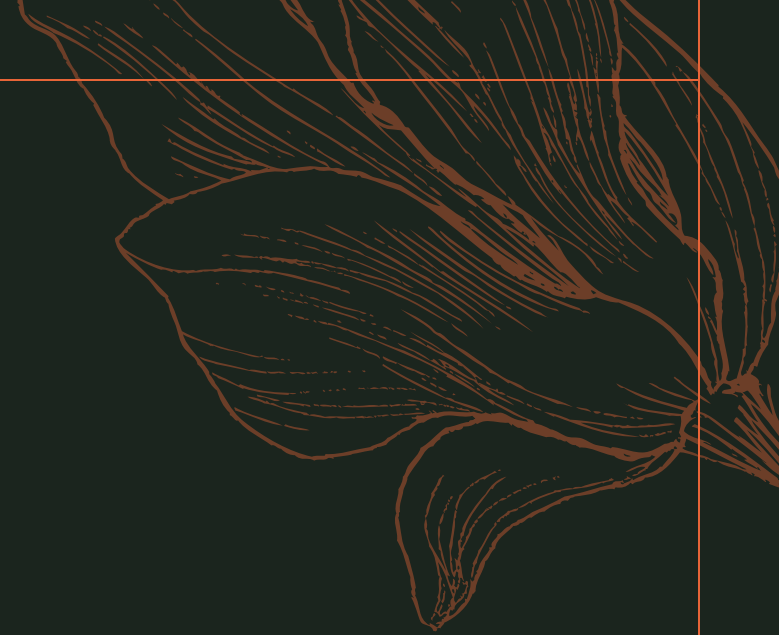
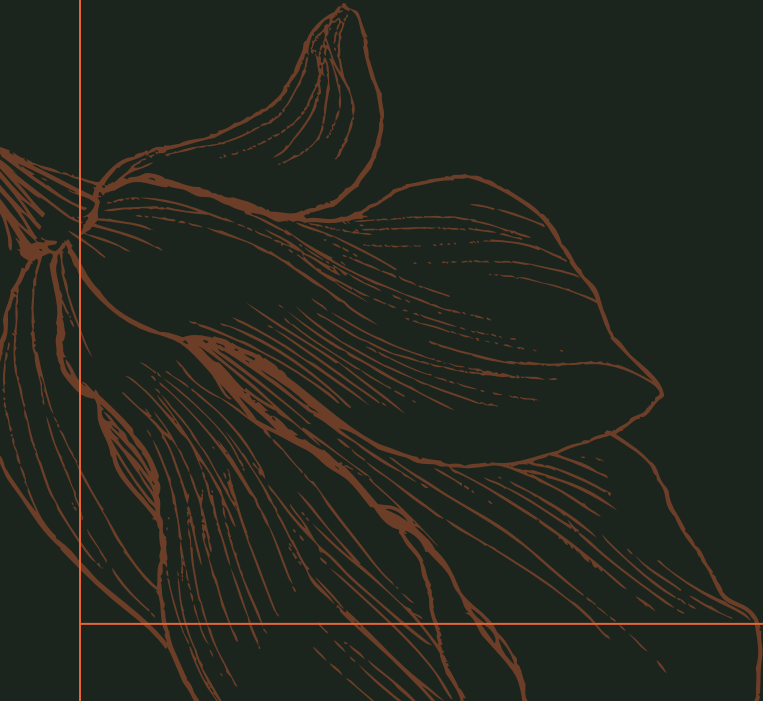


Extended E-R Features

- E-R concepts can model most database features, some aspects of a database may be more aptly expressed by certain extensions to the basic E-R model.
- Extended E-R features:
 - Specialization
 - Generalization,
 - Higher- and lower-level entity sets
 - Attribute inheritance
 - Aggregation.

Specialization

Entity-Relationship Modeling



Specialization

- An entity set may **include subgroupings of entities** that are distinct in some way from other entities in the set. It is a **Top-down design process**.
- For instance, **a subset of entities within an entity set may have attributes that are not shared by all the entities in the entity set**.
- The E-R model provides a means for representing these **distinctive entity groupings**.
- Consider an entity set person, with attributes name, street, and city. A person may be further classified as one of the following:
 - customer
 - employee

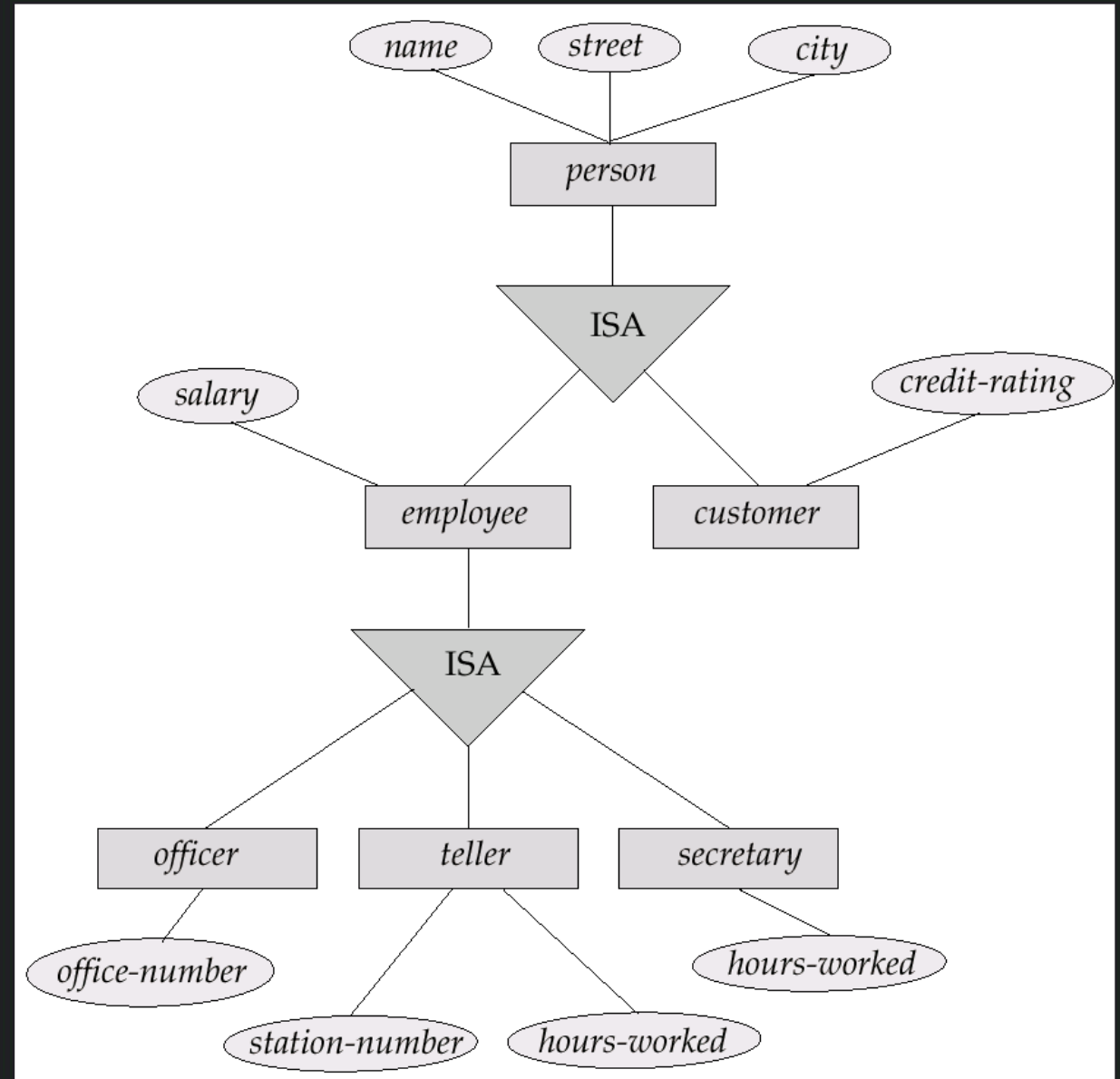
Specialization

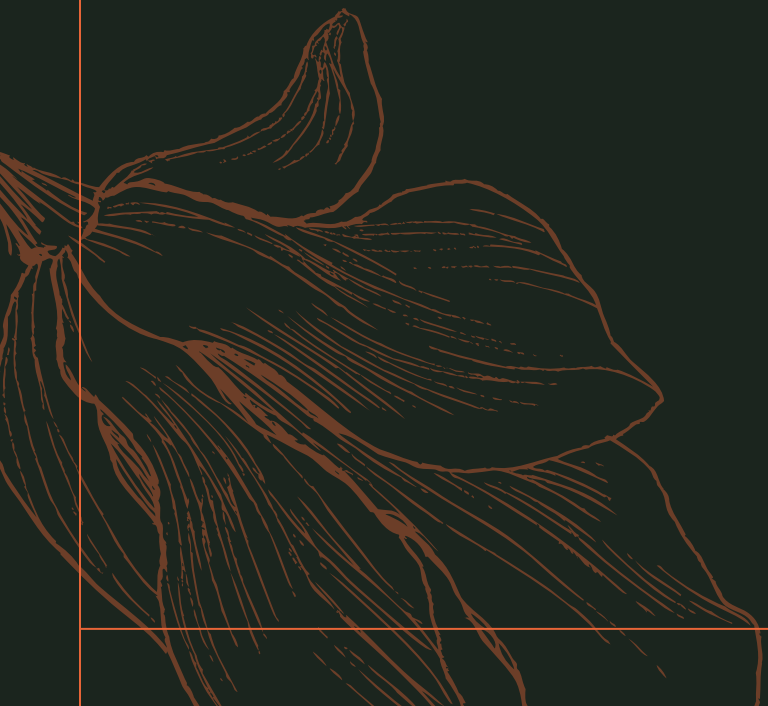
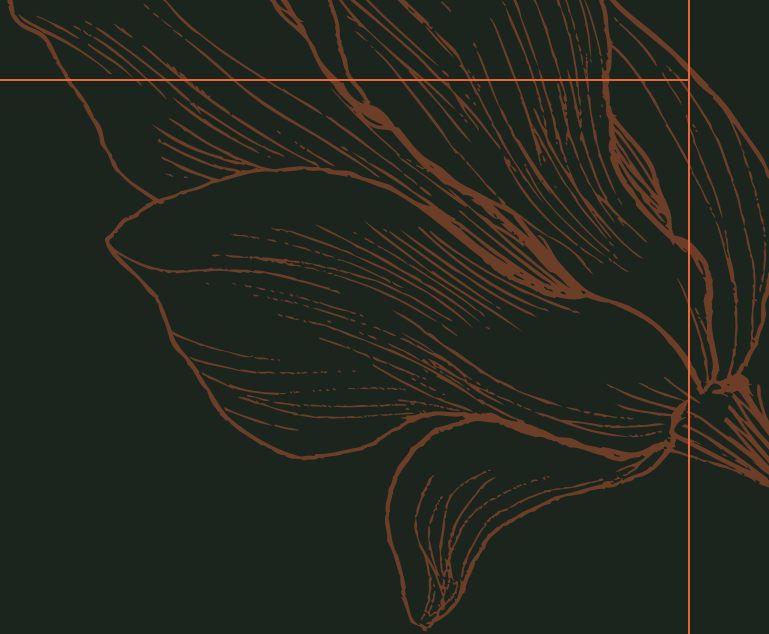
- Each of these person types is described by a set of attributes that includes all the attributes of entity set person plus possibly additional attributes.
- For example, customer entities may be described further by the attribute customer-id, whereas employee entities may be described further by the attributes employee-id and salary.
- The process of designating subgroupings within an entity set is called specialization.
- The specialization of person allows us to distinguish among persons according to whether they are employees or customers.

Specialization

- In terms of an E-R diagram, specialization is depicted by a triangle component labeled **ISA**.
- The label **ISA** stands for “is a” and represents, **for example, that a customer “is a” person.**
- The ISA relationship may also be **referred to as a superclass-subclass relationship.**
- *Higher- and lower-level entity sets are depicted as regular entity sets—that is, as rectangles containing the name of the entity set.*

Specialization & Generalization





Generalization

Entity-Relationship Modeling

Generalization

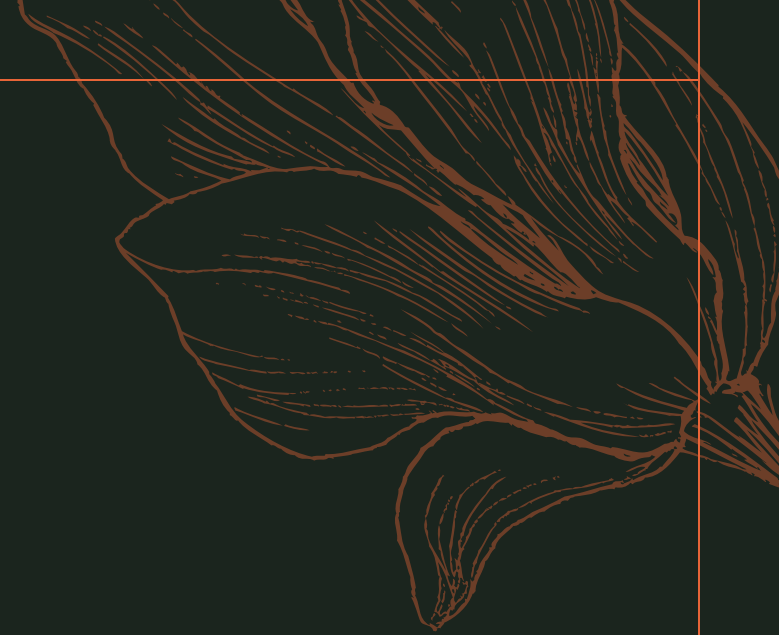
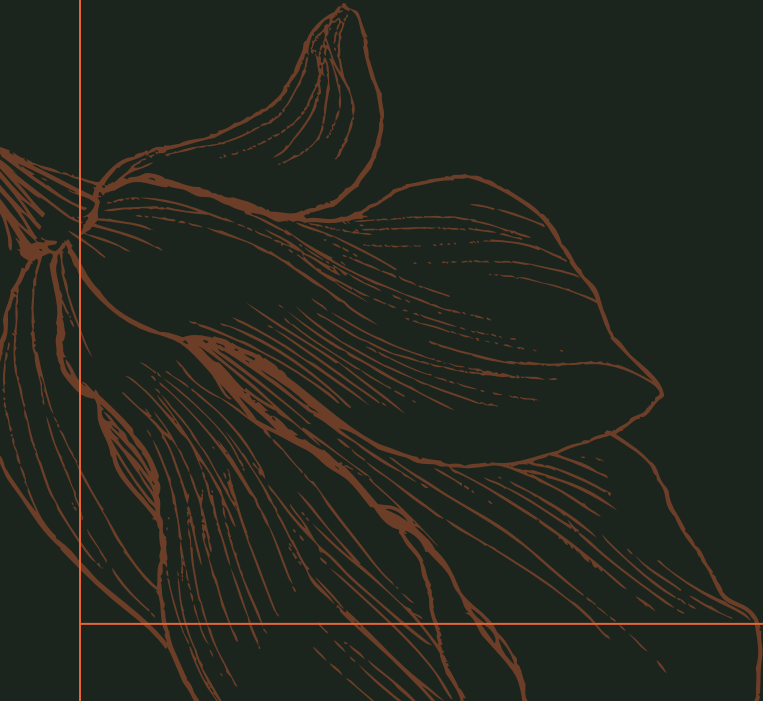
- A bottom-up design process – combine a number of entity sets that share the same features into a higher-level entity set.
- Specialization and generalization are simple inversions of each other; they are represented in an E-R diagram in the same way.
- The terms specialization and generalization are used interchangeably.

Generalization

- Can have multiple specializations of an entity set based on different features.
- E.g. permanent-employee vs. temporary-employee, in addition to officer vs. secretary vs. teller
- Each particular employee would be
 - a member of one of permanent-employee or temporary-employee,
 - and also a member of one of officer, secretary, or teller
- The ISA relationship also referred to as **superclass - subclass relationship**

Attribute Inheritance

Entity-Relationship Modeling



Attribute Inheritance

- A crucial property of the higher- and lower-level entities created by specialization and generalization is **attribute inheritance**.
- The attributes of the higher-level entity sets are said to be inherited by the lower-level entity sets.
- For example, customer and employee inherit the attributes of person. Thus, *customer is described by its name, street, and city attributes, and additionally a customer-id attribute*; employee is described by its name, street, and city attributes, and additionally employee-id and salary attributes.

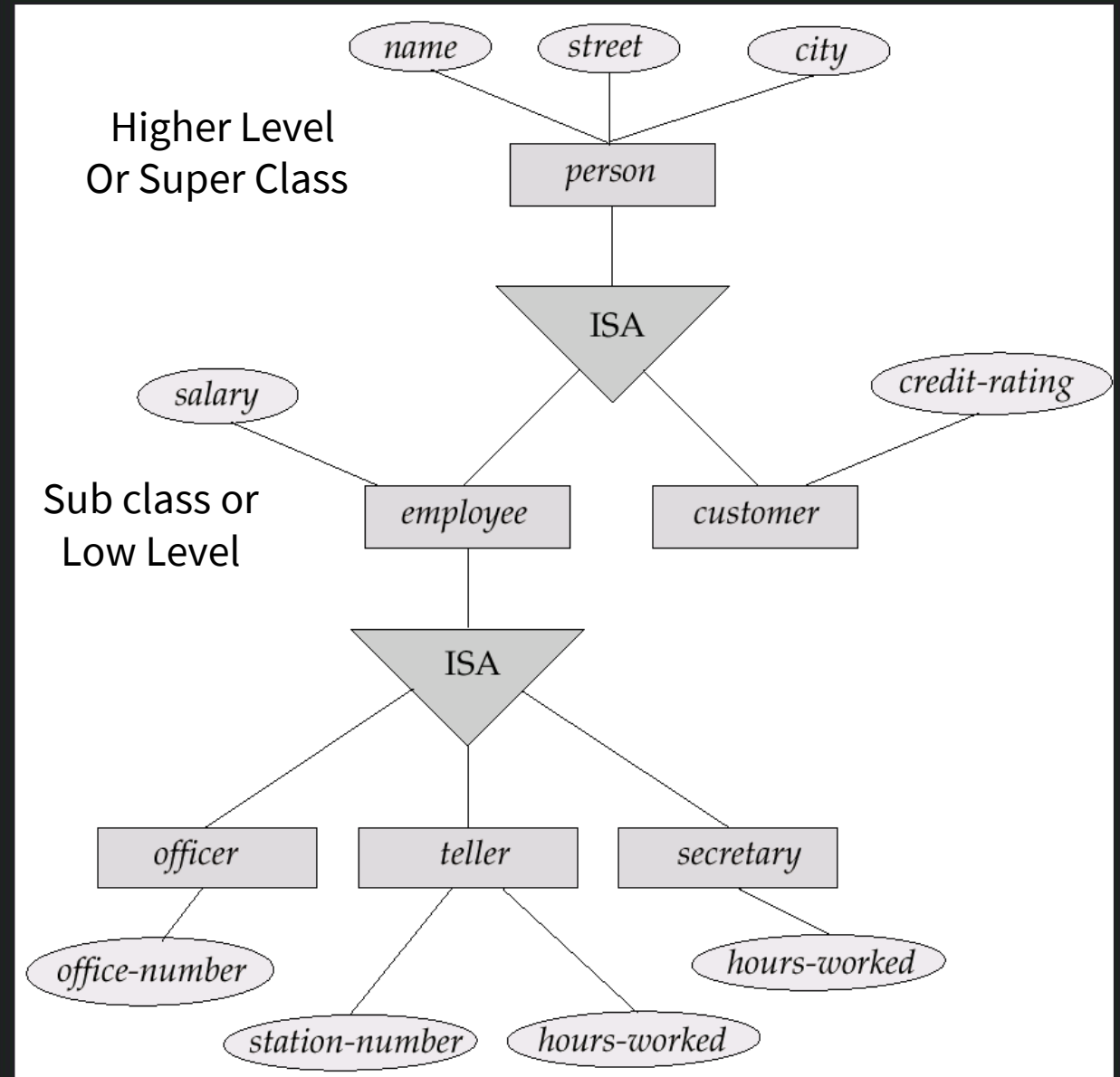
Attribute Inheritance

- A lower-level entity set (or subclass) also inherits participation in the relationship sets in which its higher-level entity (or superclass) participates.
- The officer, teller, and secretary entity sets can participate in the works-for relationship set, since the superclass employee participates in the works-for relationship.
- Attribute inheritance applies through all tiers of lower-level entity sets. The above entity sets can participate in any relationships in which the person entity set participates.

Attribute Inheritance

- Whether a given portion of an E-R model was arrived at by specialization or generalization, the outcome is basically the same:
 - A higher-level entity set with attributes and relationships that apply to all of its lower-level entity sets
 - Lower-level entity sets with distinctive features that apply only within a particular lower-level entity set

Attribute Inheritance

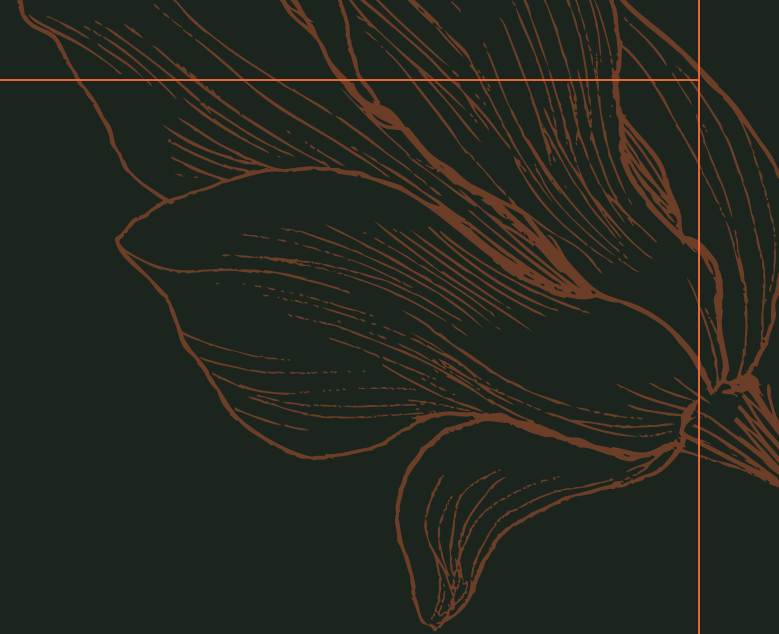
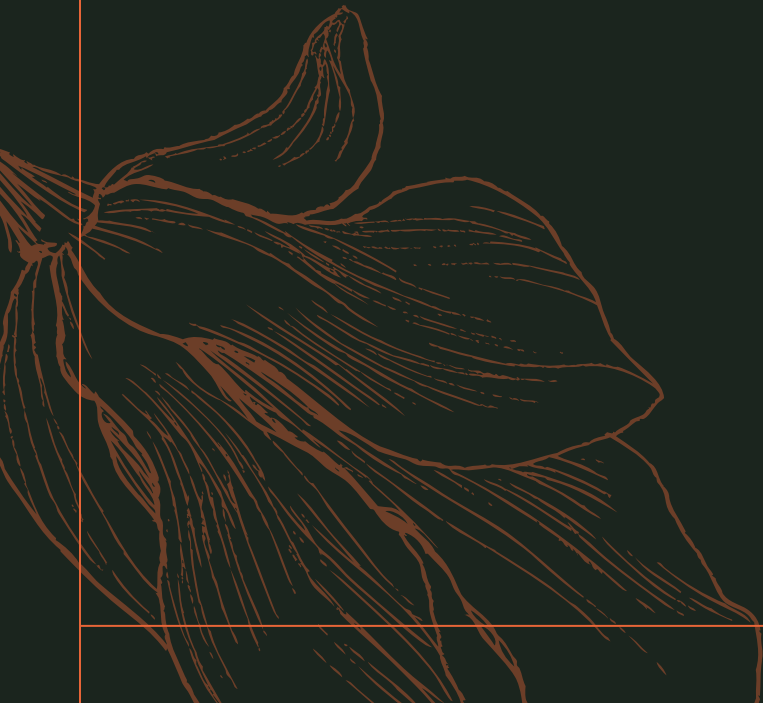


Attribute Inheritance

- Figure depicts a **hierarchy** of entity sets. In the figure, employee is a **higher-level entity set** of **person** and a **lower-level entity set** of the officer, teller, and secretary entity sets.
- In a hierarchy, a given entity set may be involved as a lower-level entity set in **only one ISA relationship**; that is, entity sets in this diagram **have only single inheritance**.
- If an entity set is a lower-level entity set in more than one ISA relationship, then the entity set has multiple inheritance, and **the resulting structure is said to be a lattice**.

Constraints

Entity-Relationship Modeling



Constraints on Generalization

- Constraint on which entities can be members of a given lower-level entity set.
 - condition-defined
 - In condition-defined lower-level entity sets, membership is evaluated on the basis of **whether or not an entity satisfies an explicit condition or predicate**.
 - For example, assume that the higher-level entity set account has the attribute account-type. All account entities are evaluated on the defining account-type attribute, based on the condition **account-type = “savings account”** are allowed to belong to the lower-level entity set person. Similarly for “checking account”. Since all the lower-level entities are evaluated on the basis of the same attribute (in this case, on account-type), this type of generalization is said to be attribute-defined **Attribute-defined**

Constraints on Generalization

- Constraint on which entities can be members of a given lower-level entity set.
 - user-defined
 - User-defined lower-level entity sets are not constrained by a membership condition; rather, the database user assigns entities to a given entity set.
 - For instance, after 3 months of employment, bank employees are assigned to one of four work teams. We therefore represent the teams as four lower-level entity sets of the higher-level employee entity set. A given employee is not assigned to a specific team entity automatically on the basis of an explicit defining condition. Instead, the user in charge of this decision makes the team assignment on an individual basis

Constraints on Generalization

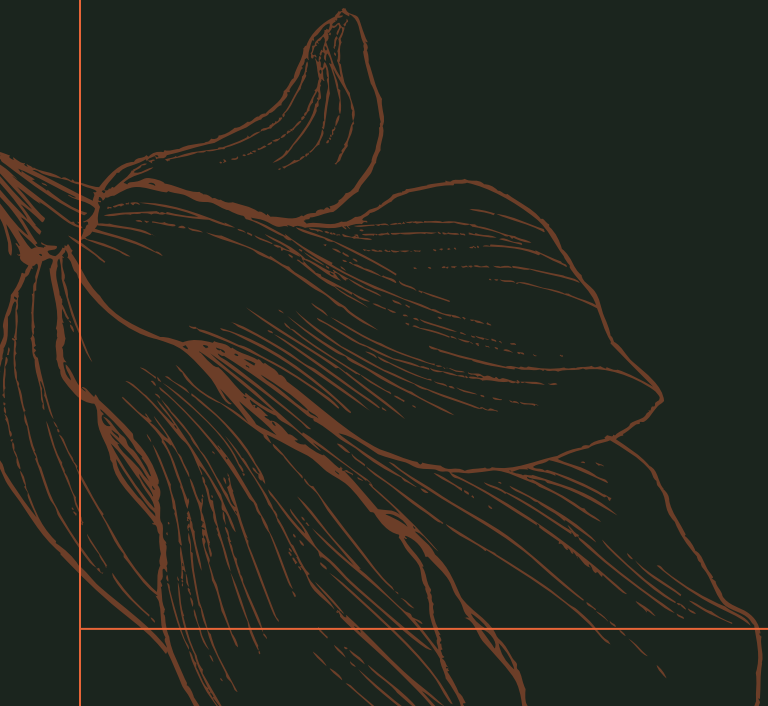
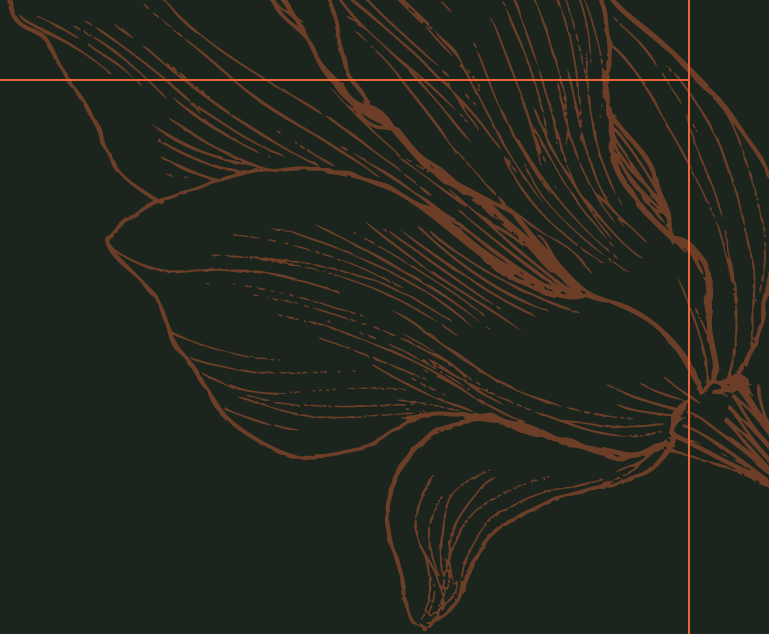
- Constraint on whether or not entities may belong to more than one lower-level entity set within a single generalization.
 - Disjoint
 - Disjointness constraint requires that **an entity belong to no more than one lower-level entity set.**
 - For example, an account entity can satisfy only one condition for the account-type attribute; an entity can be either a **savings account or a checking account, but cannot be both.**

Constraints on Generalization

- Constraint on whether or not entities may belong to more than one lower-level entity set within a single generalization.
 - Overlapping
 - Overlapping generalizations, the same entity may belong to more than one lower-level entity set within a single generalization.
 - For Example, consider the employee work team example, and assume that certain managers participate in more than one work team. A given employee may therefore appear in more than one of the team entity sets that are lower-level entity sets of employee. Thus, the generalization is overlapping.

Constraints on Generalization

- Completeness constraint -- specifies **whether or not an entity in the higher-level entity set must belong to at least one of the lower-level entity sets within a generalization.**
 - **Total** : an entity must belong to one of the lower-level entity sets
 - **Partial**: an entity need not belong to one of the lower-level entity sets

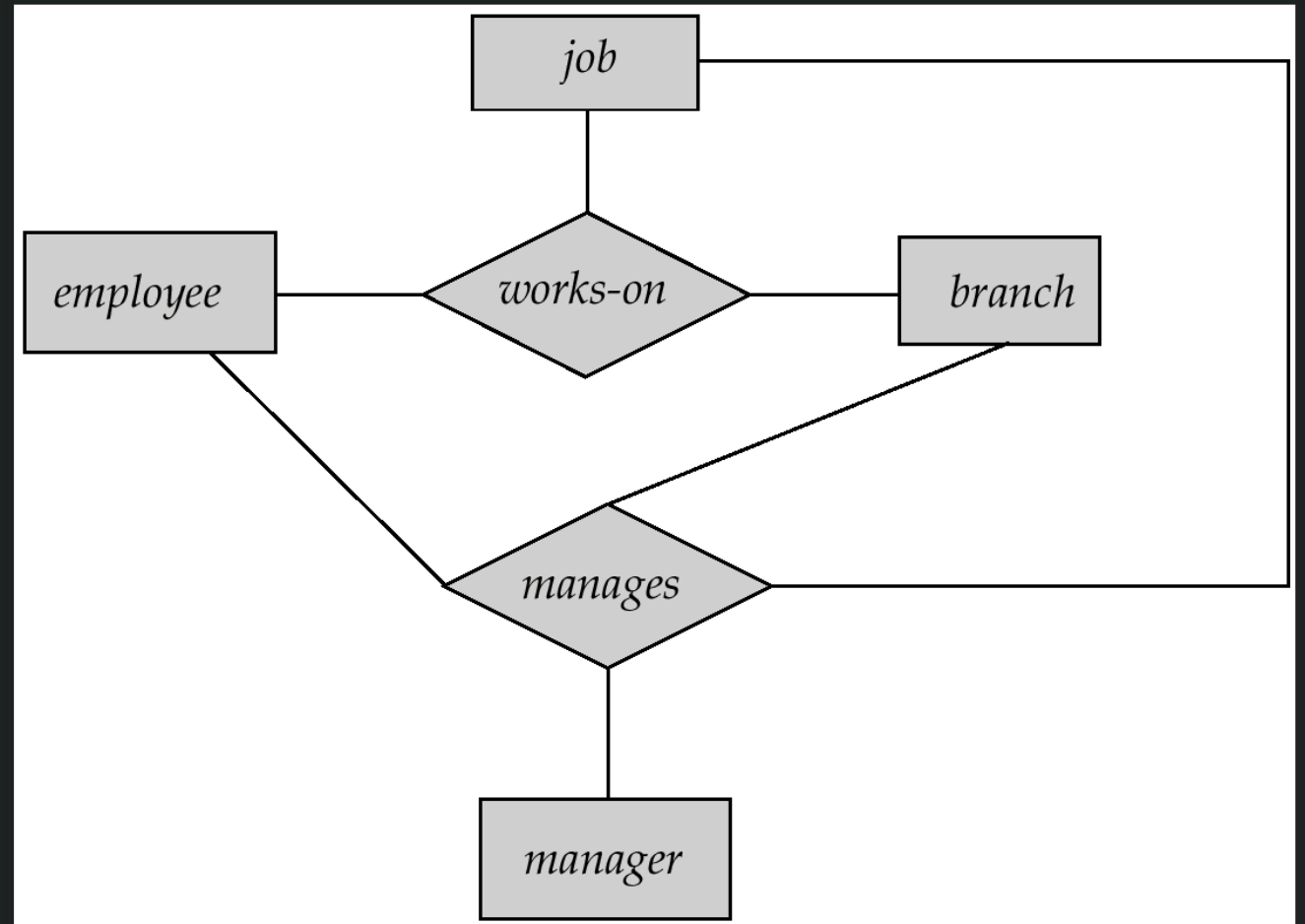


Aggregation

Entity-Relationship Modeling

Aggregation

- Consider the ternary relationship *works-on*, which we saw earlier
- Suppose we want to record managers for tasks performed by an employee at a branch



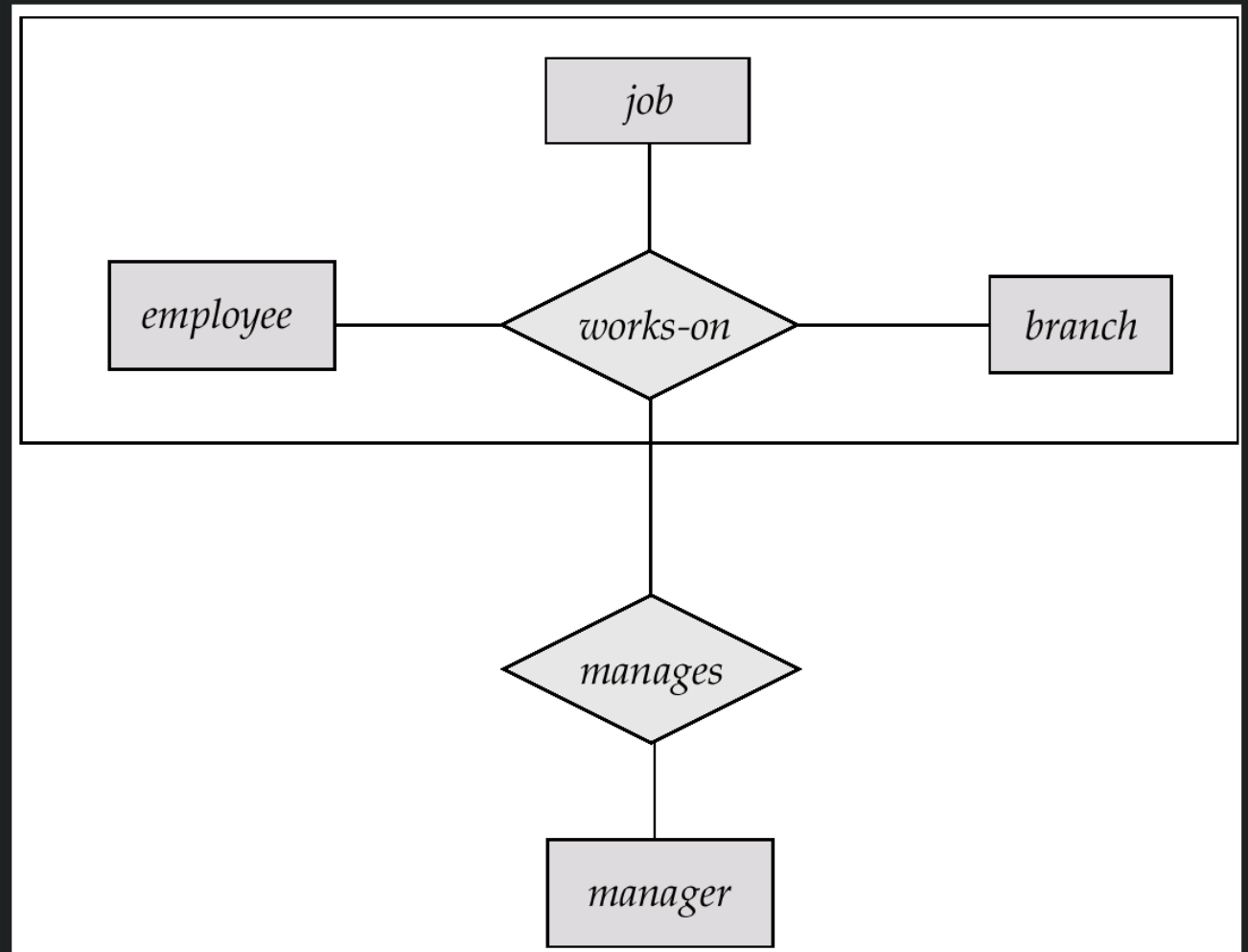
Aggregation

- Relationship sets **works-on** and **manages** **represent overlapping information**
 - Every manages relationship corresponds to a works-on relationship
 - However, some works-on relationships may not correspond to any manages relationships
 - So we can't discard the works-on relationship

Aggregation

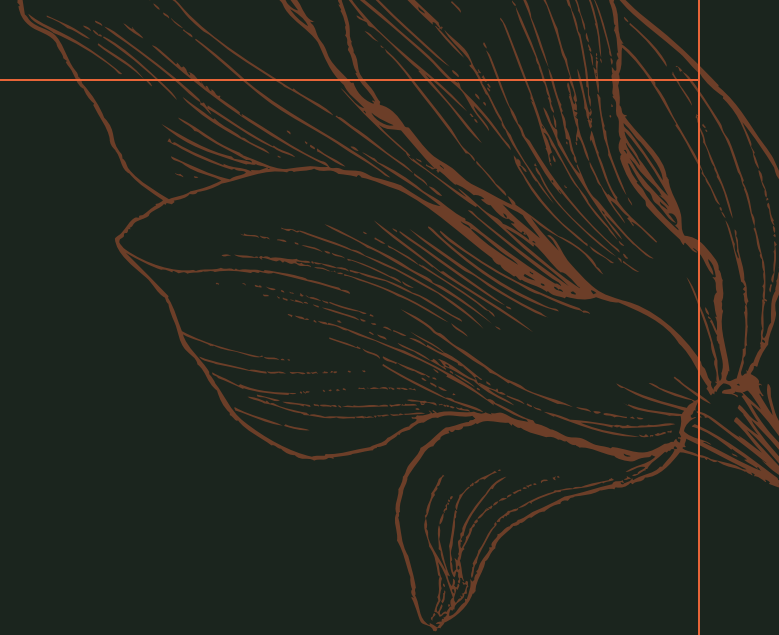
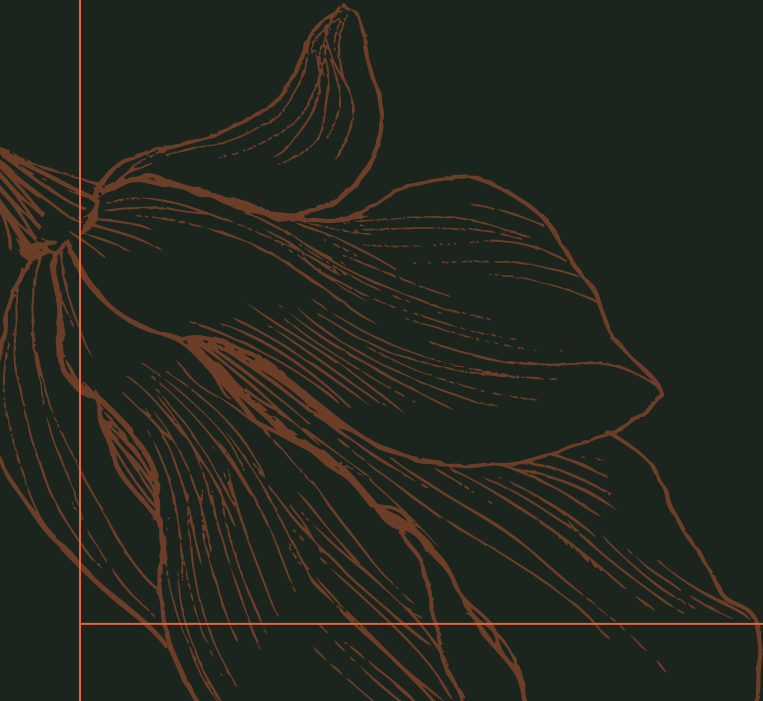
- Eliminate this redundancy via aggregation
 - Treat relationship as an abstract entity
 - Allows relationships between relationships
 - Abstraction of relationship into new entity
- Without introducing redundancy, the following diagram represents:
 - An employee works on a particular job at a particular branch
 - An employee, branch, job combination may have an associated manager

Aggregation



ER Design Issues

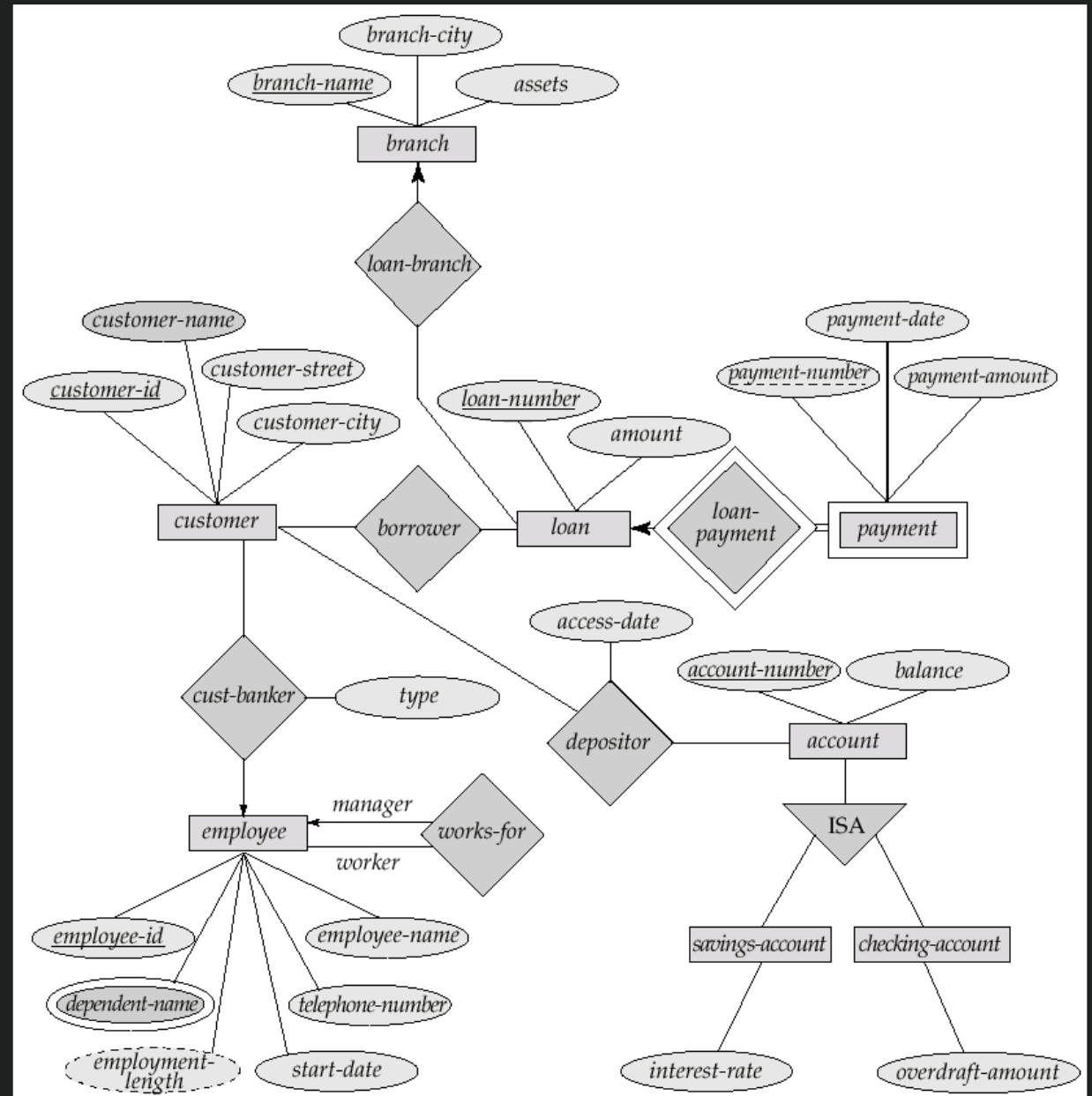
Entity-Relationship Modeling



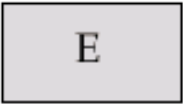
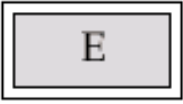
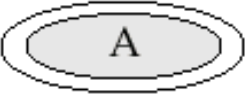
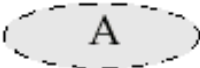
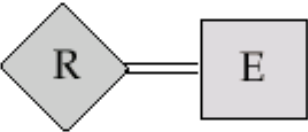
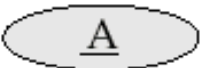
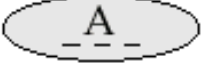
E-R Design Decisions

- The use of an attribute or entity set to represent an object.
- Whether a real-world concept is best expressed by an entity set or a relationship set.
- The use of a ternary relationship versus a pair of binary relationships.
- The use of a strong or weak entity set.
- The use of specialization/generalization – contributes to modularity in the design.
- The use of aggregation – can treat the aggregate entity set as a single unit without concern for the details of its internal structure.

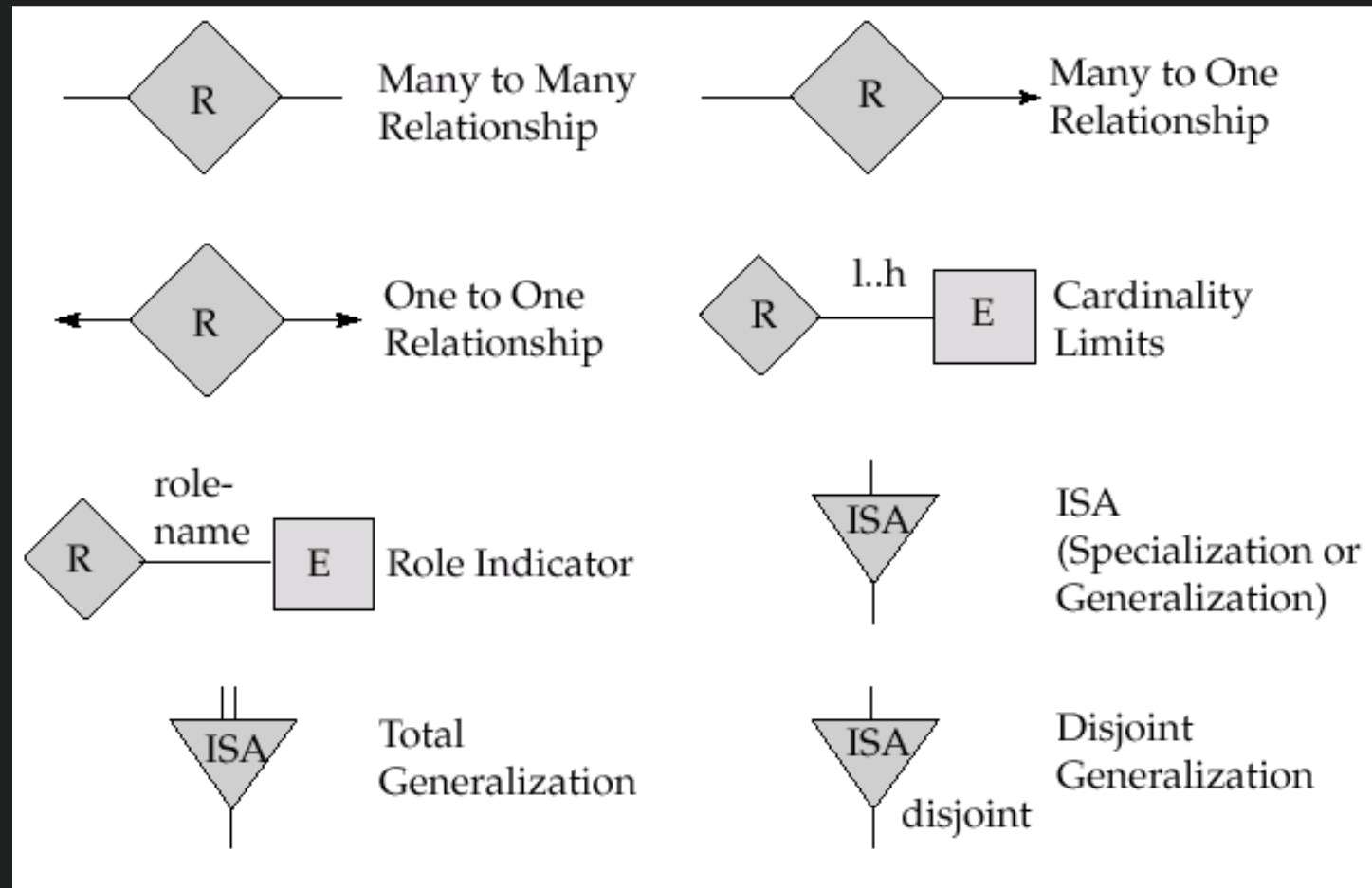
Banking Enterprise



Summary of symbols Used in E-R Notations

	Entity Set		Attribute
	Weak Entity Set		Multivalued Attribute
	Relationship Set		Derived Attribute
	Identifying Relationship Set for Weak Entity Set		Total Participation of Entity Set in Relationship
	Primary Key		Discriminating Attribute of Weak Entity Set

Summary of symbols Used in E-R Notations

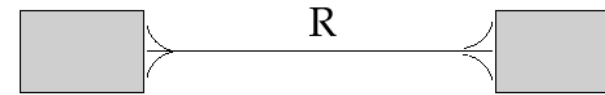
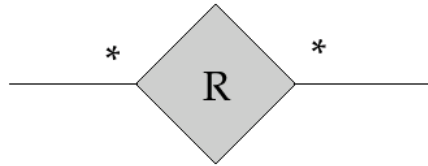


Summary of symbols Used in E-R Notations

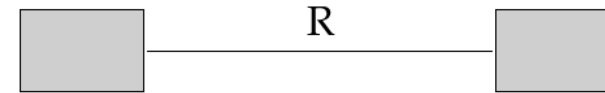
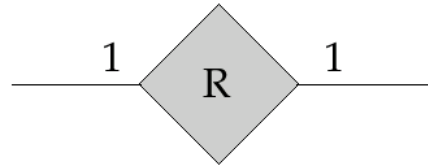
Entity set E with
attributes A1, A2, A3
and primary key A1

E	
A1	
A2	
A3	

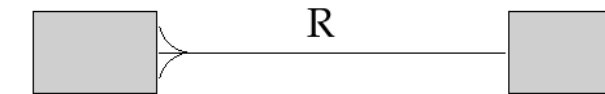
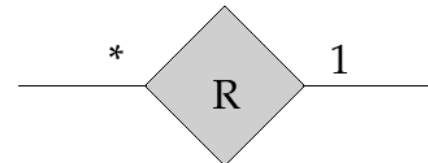
Many to Many
Relationship



One to One
Relationship

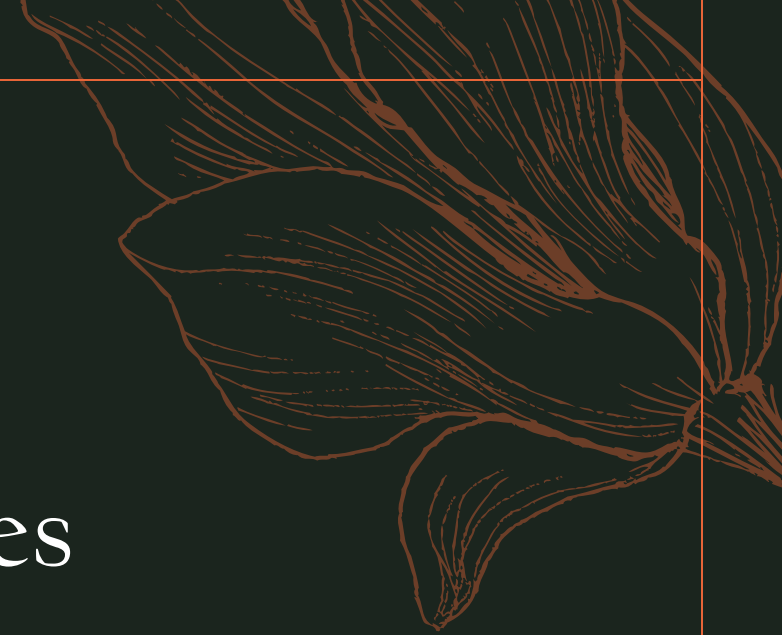
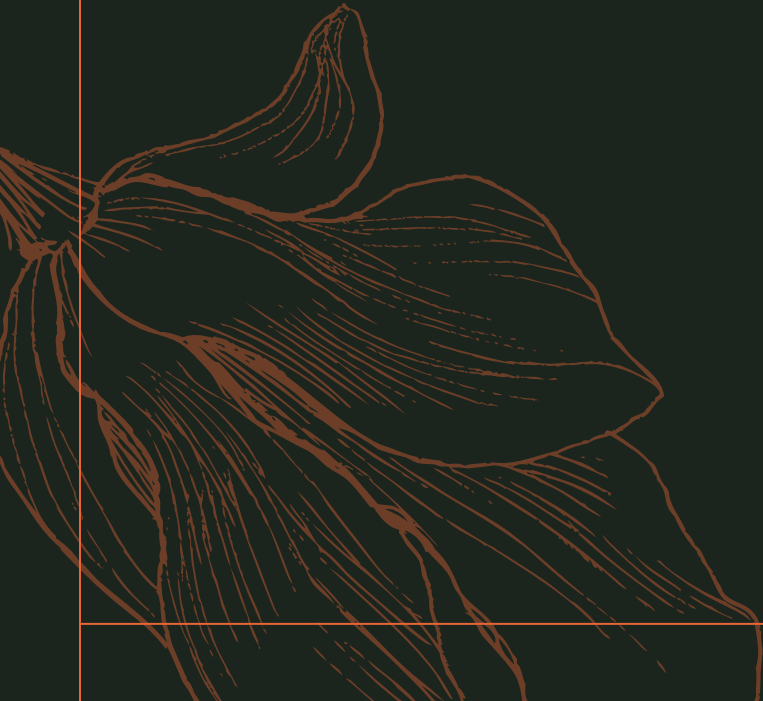


Many to One
Relationship



Reduction of ER to Tables

Entity-Relationship Modeling



Reduction of an E-R schema to Tables

- Primary keys allow entity sets and relationship sets to be expressed uniformly as tables which represent the contents of the database.
- A database which conforms to an E-R diagram can be represented by a collection of tables.
- For each entity set and relationship set there is a unique table which is assigned the name of the corresponding entity set or relationship set.
- Each table has a number of columns (generally corresponding to attributes), which have unique names.
- Converting an E-R diagram to a table format is the basis for deriving a relational database design from an E-R diagram.

Representing Entity Sets as Tables

- A strong entity set reduces to a table with the same attributes.

<i>customer-id</i>	<i>customer-name</i>	<i>customer-street</i>	<i>customer-city</i>
019-28-3746	Smith	North	Rye
182-73-6091	Turner	Putnam	Stamford
192-83-7465	Johnson	Alma	Palo Alto
244-66-8800	Curry	North	Rye
321-12-3123	Jones	Main	Harrison
335-57-7991	Adams	Spring	Pittsfield
336-66-9999	Lindsay	Park	Pittsfield
677-89-9011	Hayes	Main	Harrison
963-96-3963	Williams	Nassau	Princeton

Representing Composite Attributes

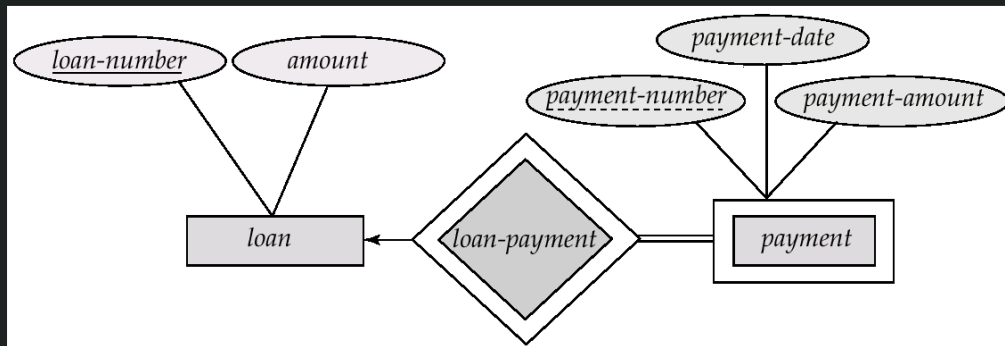
- Composite attributes are flattened out by **creating a separate attribute for each component attribute**
- E.g. given entity set customer with composite attribute name with component attributes first-name and last-name the table corresponding to the entity set has two attributes
name.first-name and name.last-name

Representing Multivalued Attribute

- A multivalued attribute M of an entity E is represented by a separate table EM
 - Table EM has attributes corresponding to the primary key of E and an attribute corresponding to multivalued attribute M
 - E.g. Multivalued attribute phone_number of customer is represented by a table
`customer-phone_number( customer_name, phonenumber)`
 - Each value of the multivalued attribute maps to a separate row of the table EM
 - E.g., an employee entity with primary key John and dependents Johnson and Johndotir maps to two rows:
`(John, Johnson)` and `(John, Johndotir)`

Representing Weak Entity Sets

- A weak entity set becomes a table that includes a column for the primary key of the identifying strong entity set



<i>loan-number</i>	<i>payment-number</i>	<i>payment-date</i>	<i>payment-amount</i>
L-11	53	7 June 2001	125
L-14	69	28 May 2001	500
L-15	22	23 May 2001	300
L-16	58	18 June 2001	135
L-17	5	10 May 2001	50
L-17	6	7 June 2001	50
L-17	7	17 June 2001	100
L-23	11	17 May 2001	75
L-93	103	3 June 2001	900
L-93	104	13 June 2001	200

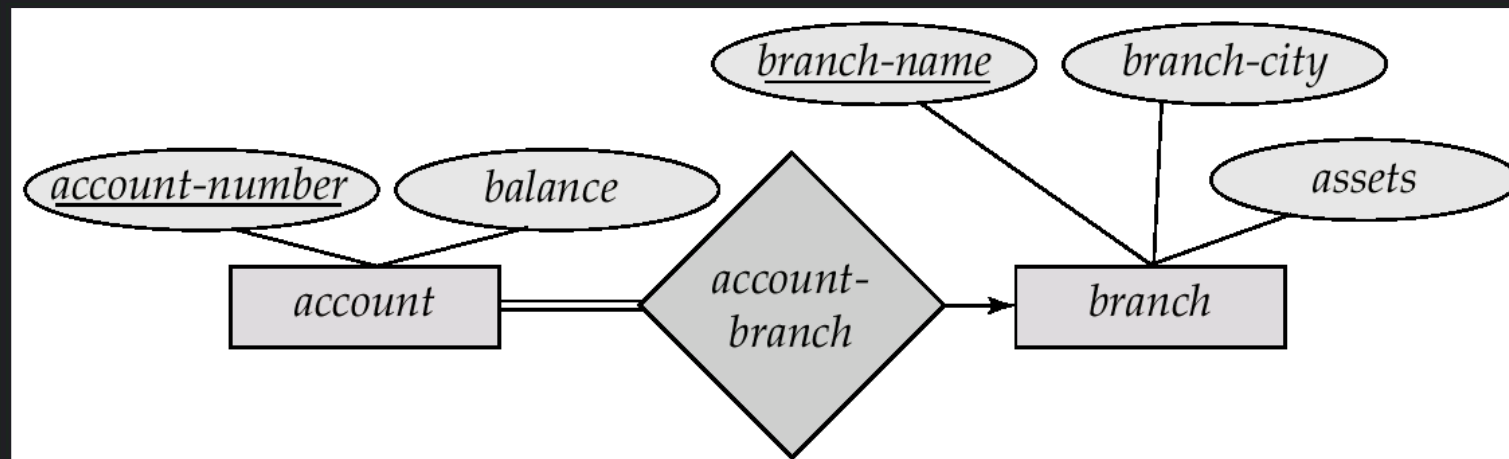
Representing Relationship Sets as Tables

- A many-to-many relationship set is represented as a table with columns for the primary keys of the two participating entity sets, and any descriptive attributes of the relationship set.
- E.g.: table for relationship set borrower

<i>customer-id</i>	<i>loan-number</i>
019-28-3746	L-11
019-28-3746	L-23
244-66-8800	L-93
321-12-3123	L-17
335-57-7991	L-16
555-55-5555	L-14
677-89-9011	L-15
963-96-3963	L-17

Redundancy of Tables

- Many-to-one and one-to-many relationship sets **that are total on the many-side** can be represented by adding an **extra attribute to the many side**, containing the primary key of the one side
- E.g.: Instead of creating a table for relationship account-branch, add an attribute branch to the entity set account



Redundancy of Tables

- For **one-to-one** relationship sets, **either side can be chosen to act as the “many” side**
 - That is, extra attribute can be added to either of the tables corresponding to the two entity sets
- If **participation is partial** on the many side, replacing a table by an **extra attribute in the relation corresponding to the “many” side** could result in null values

Redundancy of Tables

- If participation is partial on the many side, replacing a table by an extra attribute in the relation corresponding to the “many” side could result in null values
- The table corresponding to a relationship set linking a weak entity set to its identifying strong entity set is redundant.
- E.g. The payment table already contains the information that would appear in the loan-payment table (i.e., the columns loan-number and payment-number).

Representing Specialization as Tables

- Method 1:
 - Form a table for the higher level entity
 - Form a table for each lower level entity set, include primary key of higher level entity set and local attributes

table	table attributes
person	name, street, city
customer	name, credit-rating
employee	name, salary

- Drawback: getting information about, e.g., employee requires accessing two tables

Representing Specialization as Tables

- Method 2:
- Form a table for each entity set with all local and inherited attributes

table	table attributes
person	name, street, city
customer	name, street, city, credit-rating
employee	name, street, city, salary

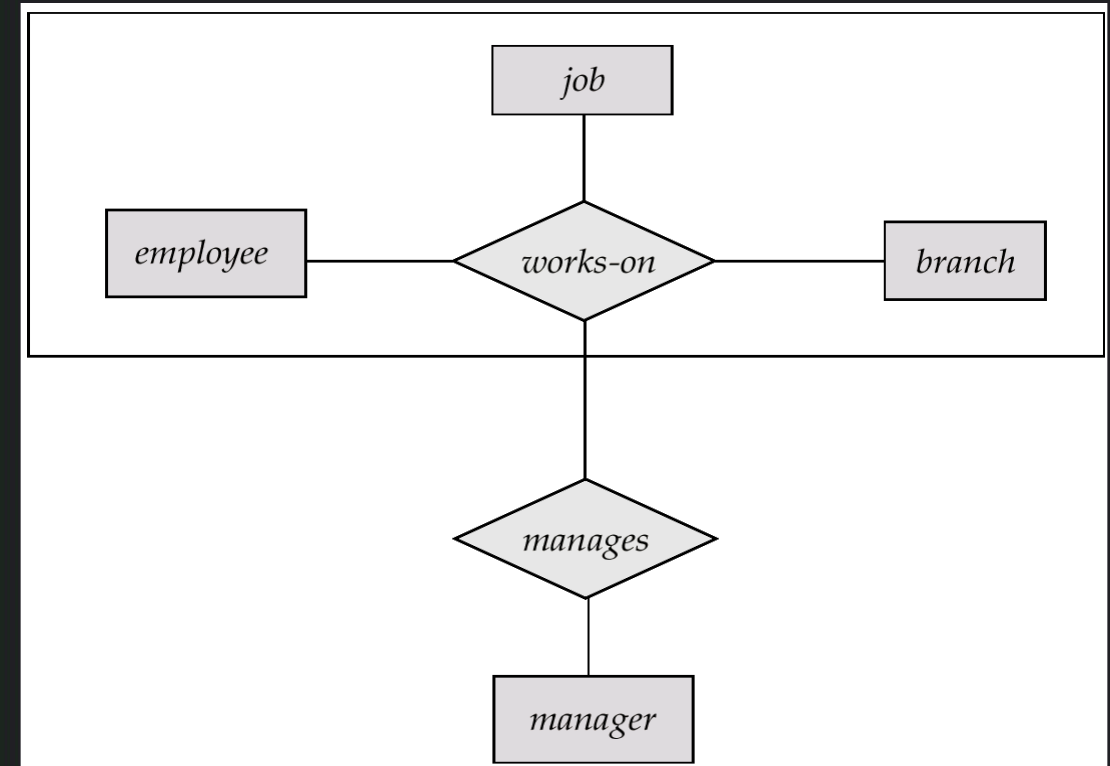
- If specialization is total, table for generalized entity (person) not required to store information
- Can be defined as a “view” relation containing union of specialization tables
- But explicit table may still be needed for foreign key constraints
- **Drawback:** street and city may be stored redundantly for persons who are both customers and employees

Relations Corresponding to Aggregation

- To represent aggregation, create a table containing
 - primary key of the aggregated relationship,
 - the primary key of the associated entity set
 - Any descriptive attributes

Relations Corresponding to Aggregation

- E.g. to represent aggregation manages between relationship works-on and entity set manager, create a table **manages(employee-id, branch-name, title, manager-name)**
- Table works-on is redundant provided we are willing to store null values for attribute manager-name in table manages



Relations Corresponding to Aggregation

- E.g. to represent aggregation manages between relationship works-on and entity set manager, create a table
manages(employee-id, branch-name, title, manager-name)
- Table works-on is redundant provided we are willing to store null values for attribute manager-name in table manages

Existence Dependencies

- If the existence of entity x depends on the existence of entity y, then **x is said to be existence dependent on y**.
- y is a **dominant entity** (in example below, loan)
- x is a **subordinate entity** (in example below, payment)



If a loan entity is deleted, then all its associated payment entities must be deleted also.

Thank you

Module 2

**BCSE302L – Database
Systems**

